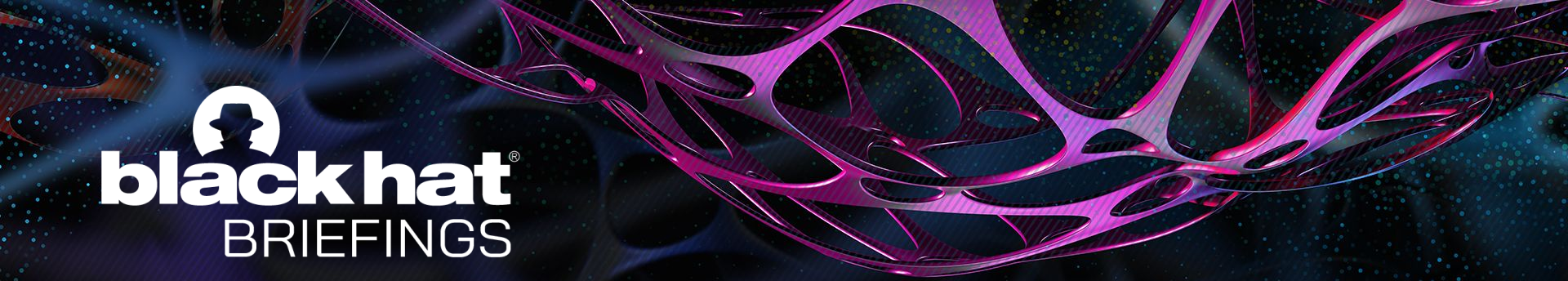


Racing for Privilege

Leaking memory on any Intel processor with a microarchitectural race condition

Sandro Rüegge & Johannes Wikner

Sandro
PhD student

Wanted to defend,
ended up attacking.

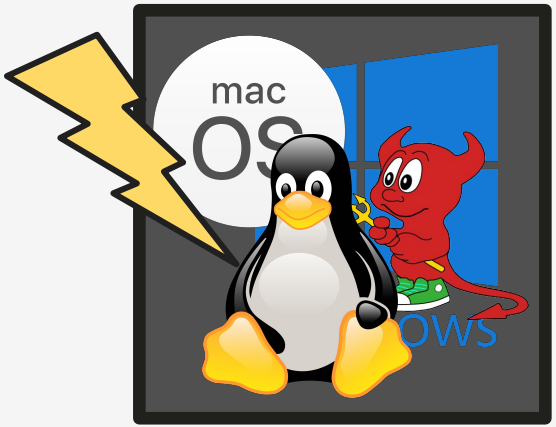
Kaveh
Professor

Johannes
PhD Graduate
Speculative execution vulnerabilities
Retbleed, Phantom (e.g., Branch Type Confusion), Inception (SRSO)

ETH zürich | COMSEC

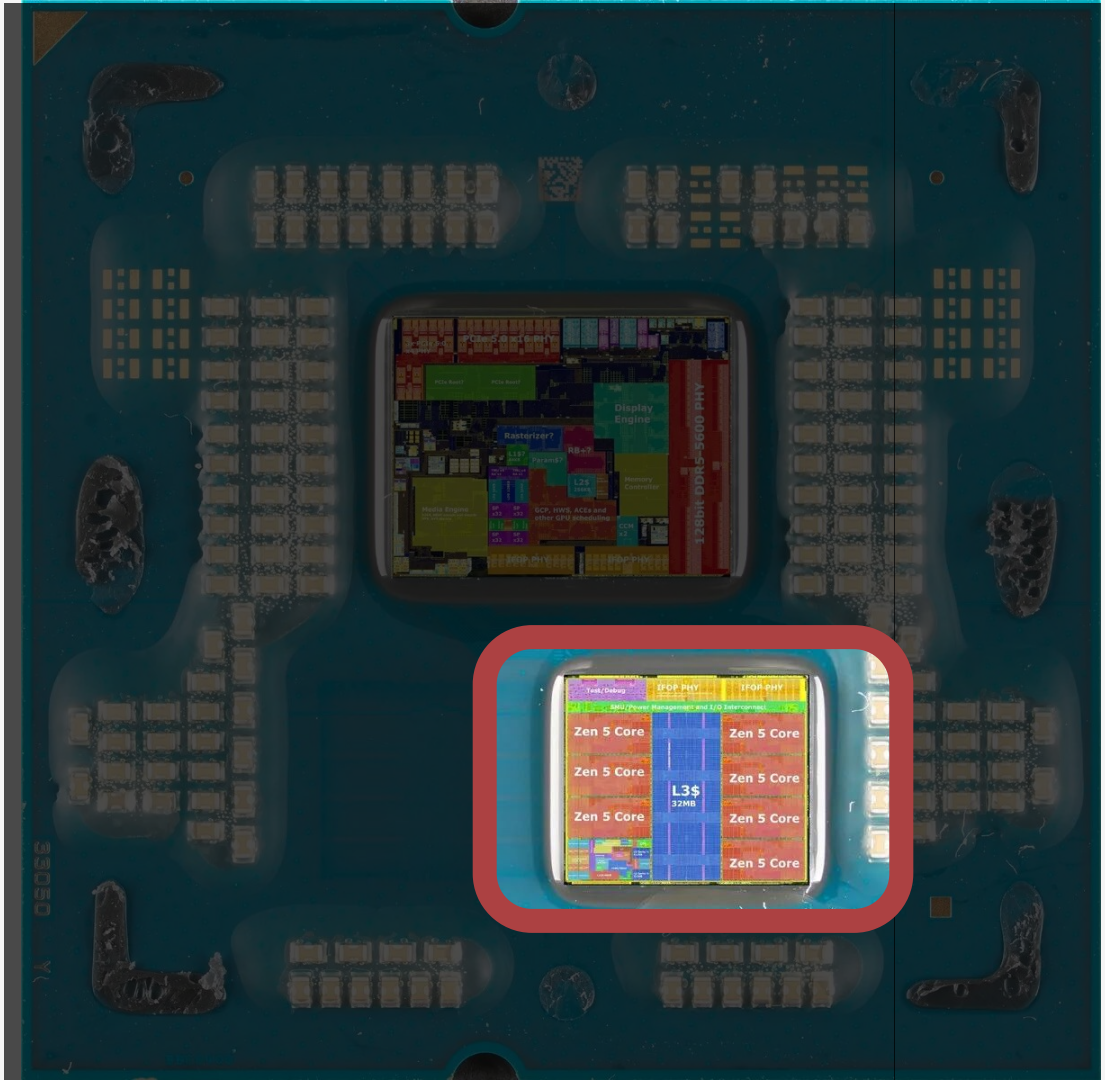
back in 2017…

A new paradigm exploitation …

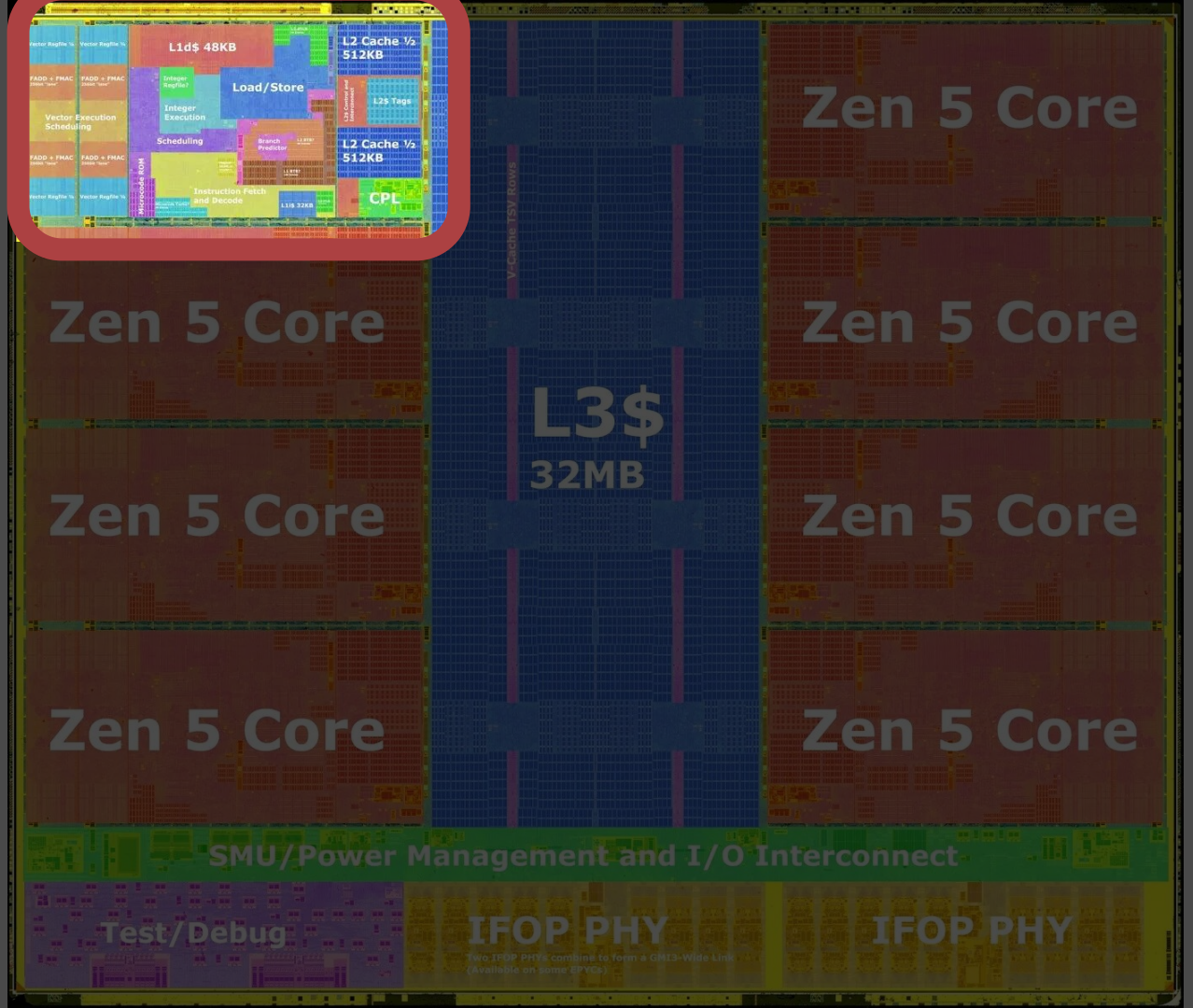


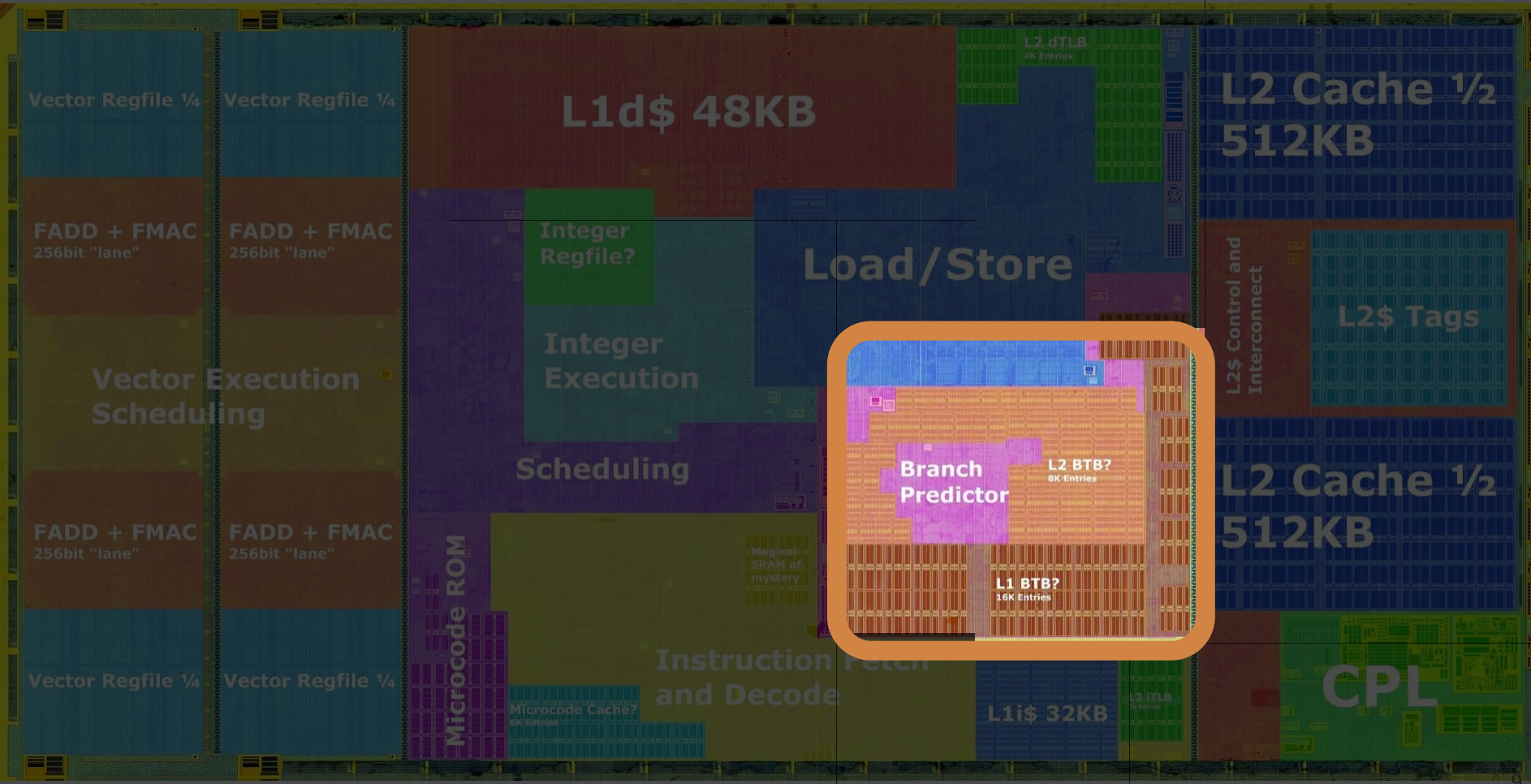

?
Xen Project
arm
AMD
amazon
Microsoft
intel

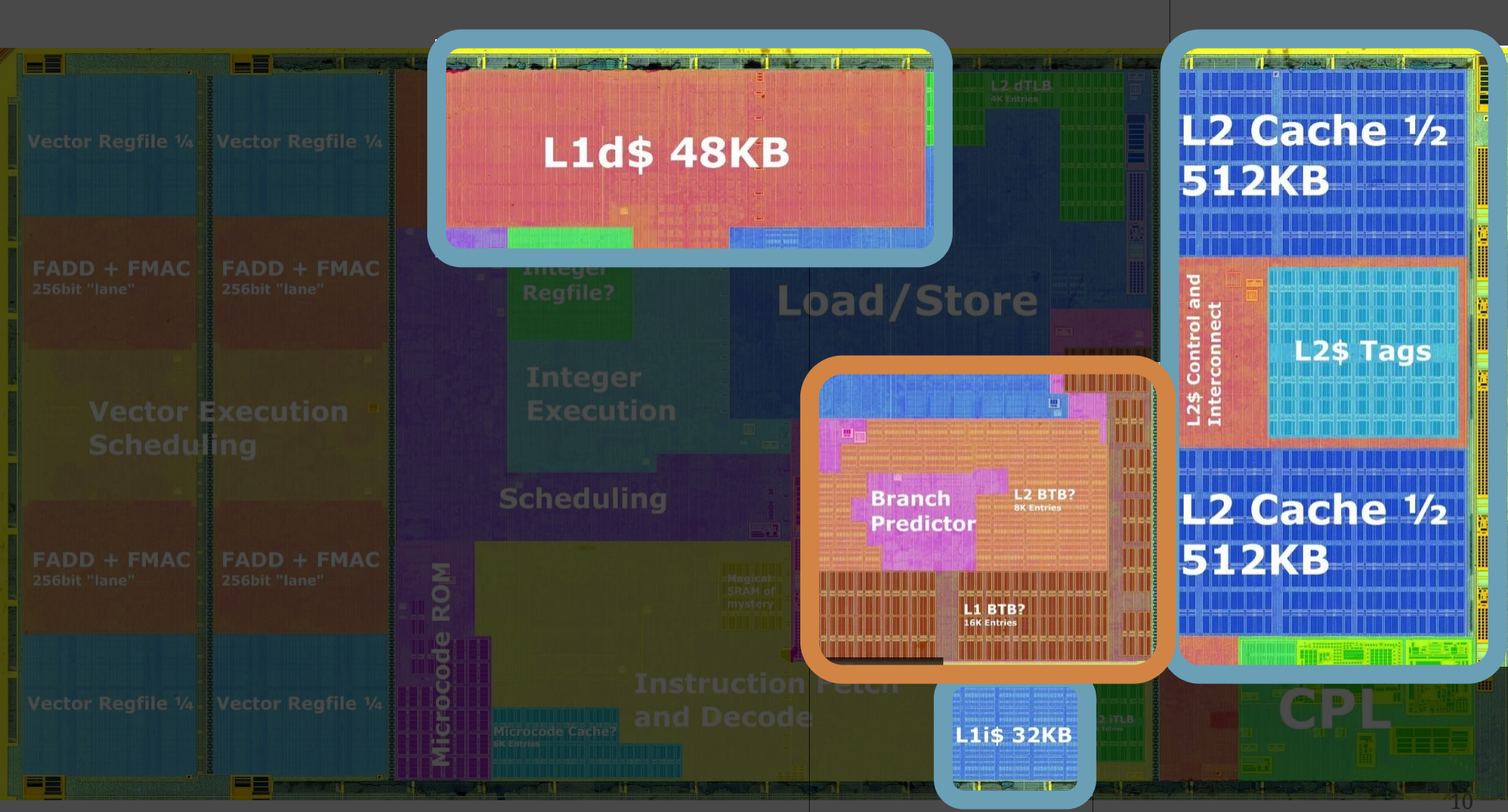
SPECTRE

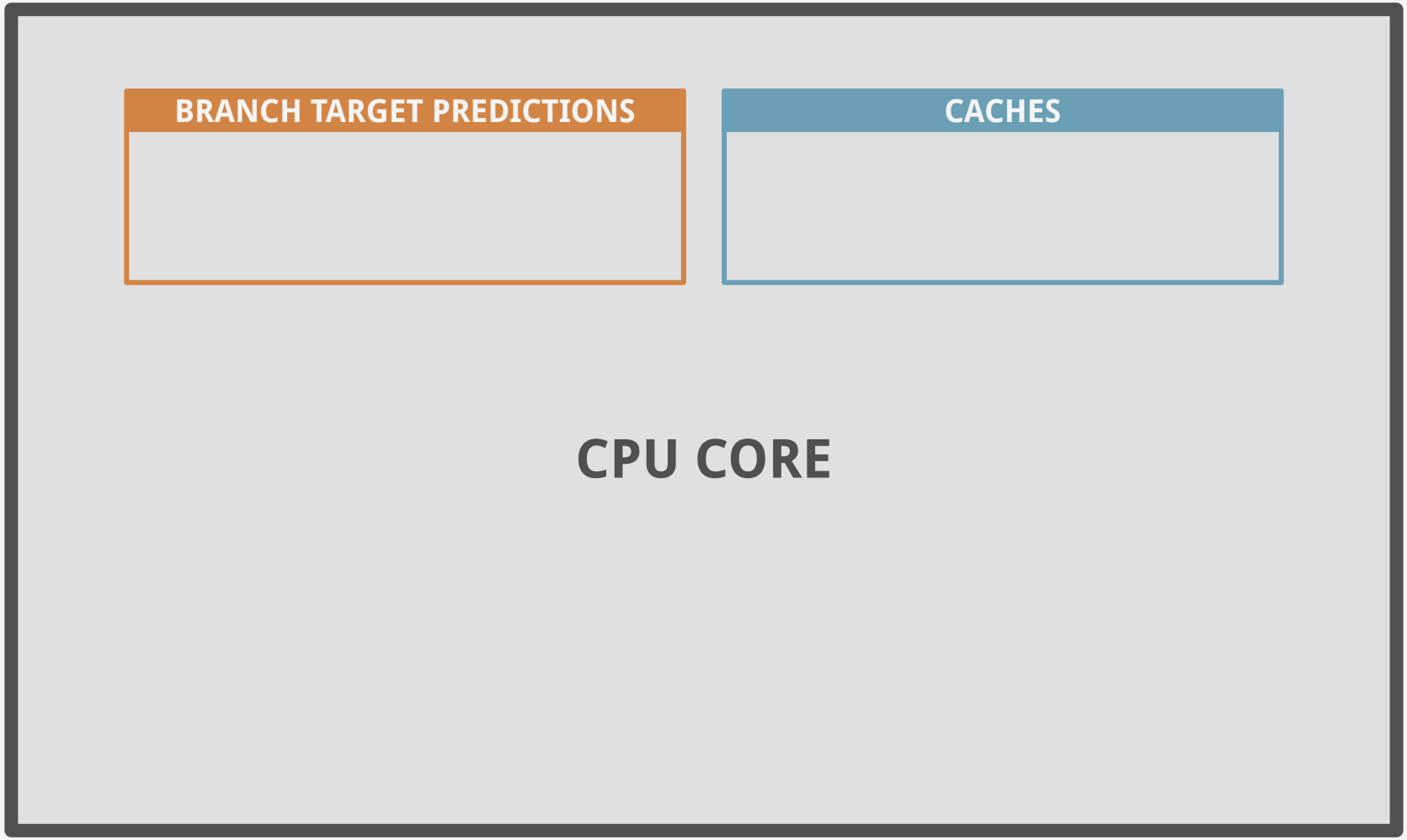


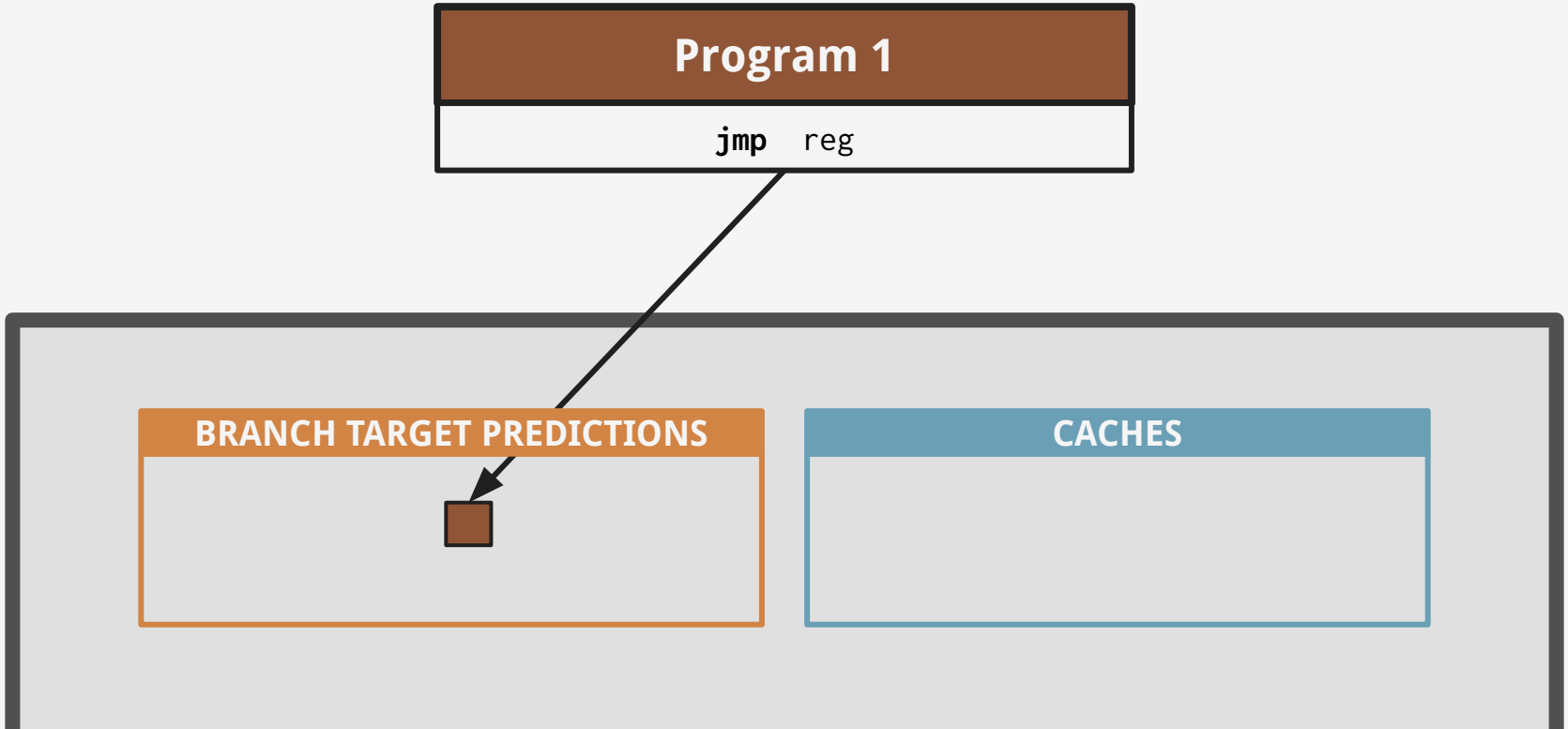
Hilbert Hagedoorn
Oct. 2024
AMD Ryzen 9000 Die Shots gets Annotated In Detail
guru3d.com/story/amd-ryzen-9000-die-shots-gets-annotated-in-detail/

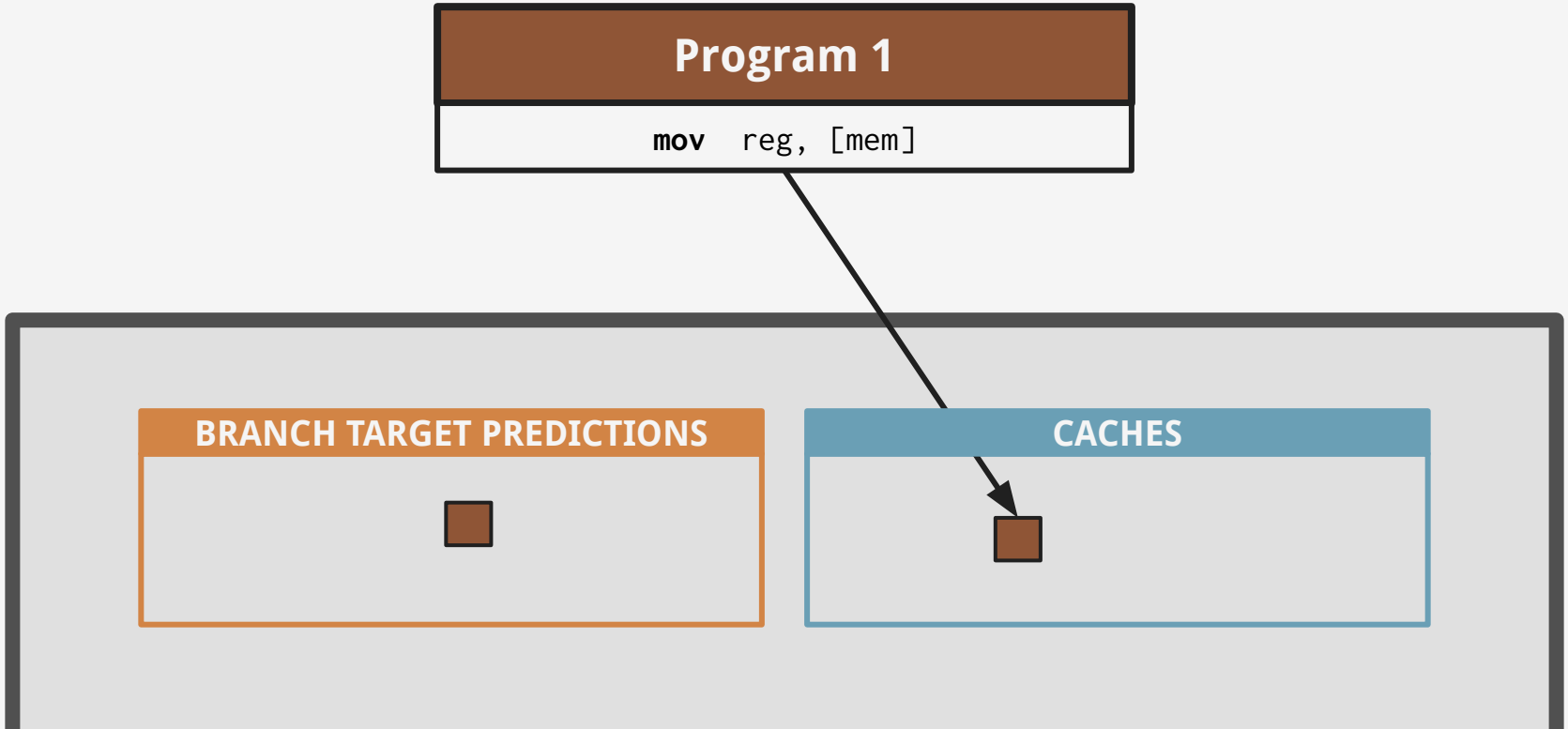

L1d$ 48KB
L2 Cache ½
512KB
Load/Store
Integer Execution
Vector Execution Scheduling
Scheduling
Branch Predictor
L2 Cache ½
512KB
CPL
Instruction Fetch and Decode
L1i$ 32KB
Zen 5 Core
Zen 5 Core
Zen 5 Core
Zen 5 Core
Zen 5 Core
Zen 5 Core
L3$
32MB
SMU/Power Management and I/O Interconnect
Test/Debug
IFOP PHY
IFOP PHY

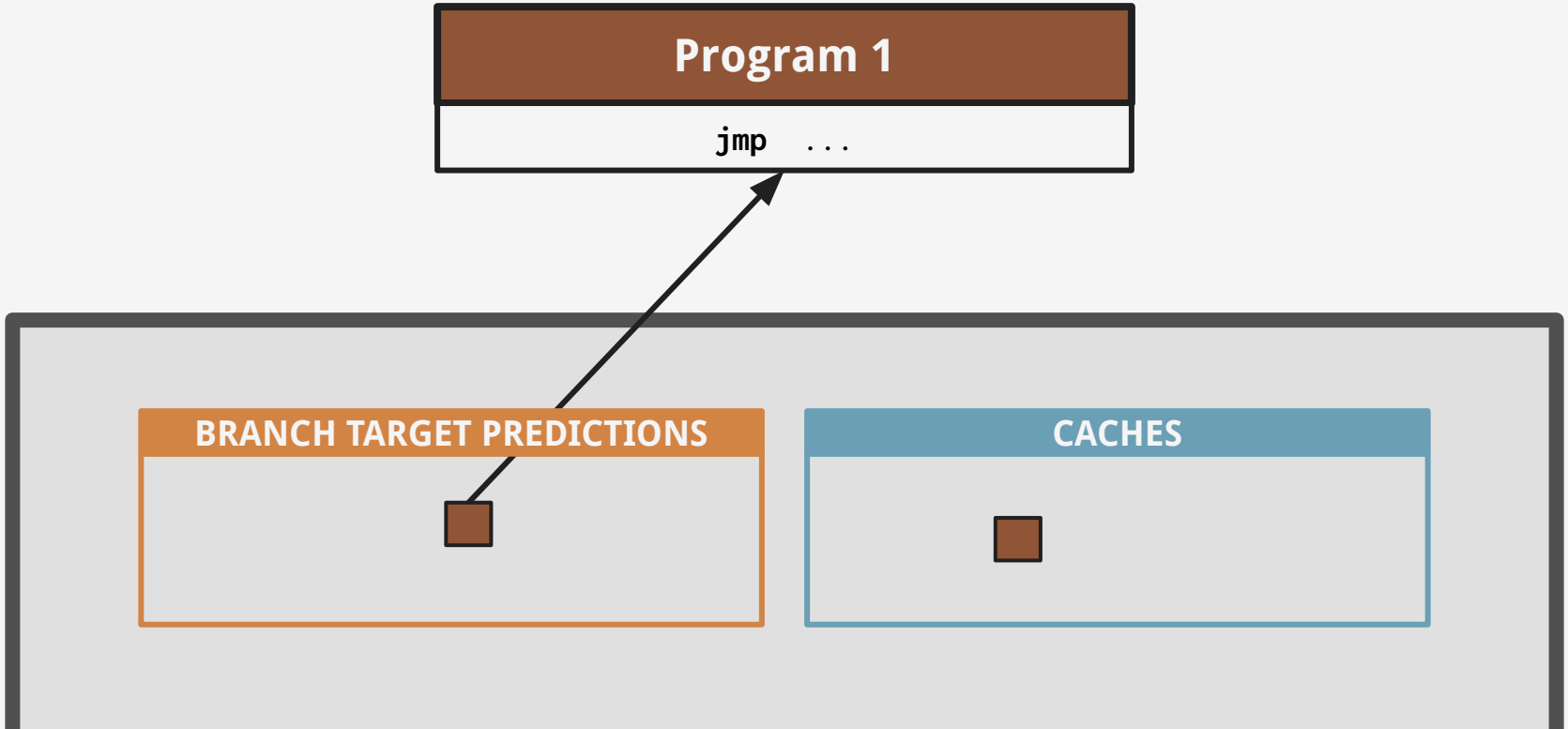

Vector Regfile ¼
Vector Regfile ¼
FADD + FMAC
256bit "lane"
FADD + FMAC
256bit "lane"
Vector Execution
Scheduling
FADD + FMAC
256bit "lane"
FADD + FMAC
256bit "lane"
Vector Regfile ¼
Vector Regfile ¼
L1d$ 48KB
L2 dTLB
4K Entries
Integer
Regfile?
Load/Store
Integer
Execution
Scheduling
Microcode ROM
Magical
SRAM of
mystery
Instruction Fetch
and Decode
Microcode Cache?
6K Entries
L1i$ 32KB
L2 iTLB
Branch
Predictor
L2 BTB?
8K Entries
L1 BTB?
16K Entries
L2 Cache ½
512KB
L2$ Control and
Interconnect
L2$ Tags
L2 Cache ½
512KB
CPL


L1d$ 48KB
L2 Cache ½
512KB
L2$ Control and Interconnect
L2$ Tags
L2 Cache ½
512KB
Branch Predictor
L2 BTB?
8K Entries
L1 BTB?
16K Entries
L1i$ 32KB
Vector Regfile ¼
FADD + FMAC
256bit "lane"
Vector Execution Scheduling
Load/Store
Integer Execution
Scheduling
Microcode ROM
Instruction Fetch and Decode
CPL


BRANCH TARGET PREDICTIONS
CACHES
CPU CORE

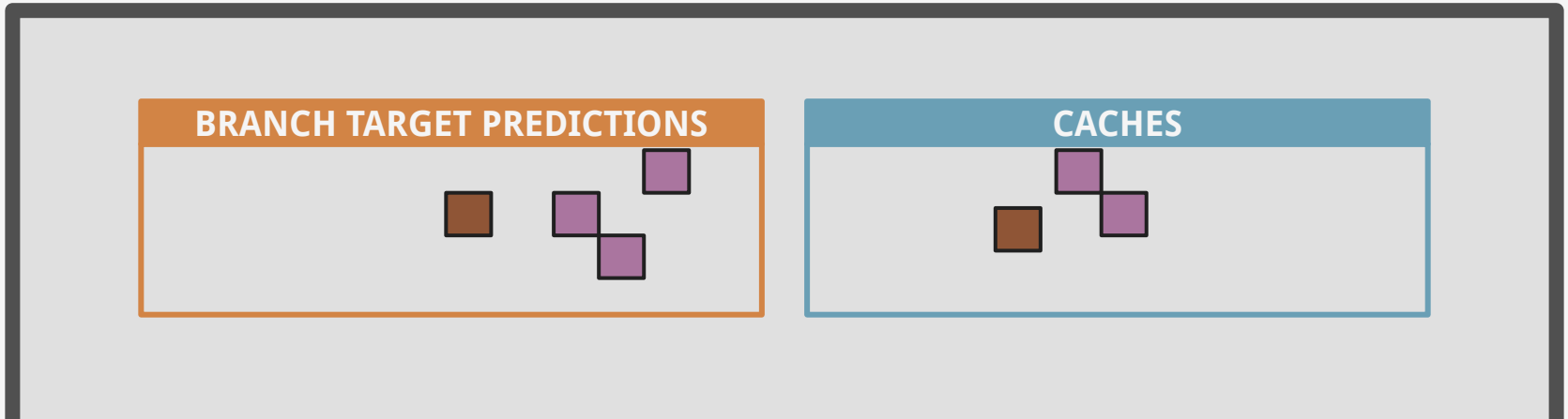

Program 1
jmp reg
BRANCH TARGET PREDICTIONS
CACHES


Program 1
mov reg, [mem]
BRANCH TARGET PREDICTIONS
CACHES


Program 1
jmp ...
BRANCH TARGET PREDICTIONS
CACHES

Program 2

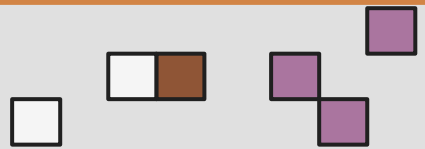
```
jmp… mov… mov… jmp… etc
```



OS/kernel (ring 0)

`jmp… mov… mov… jmp… etc`

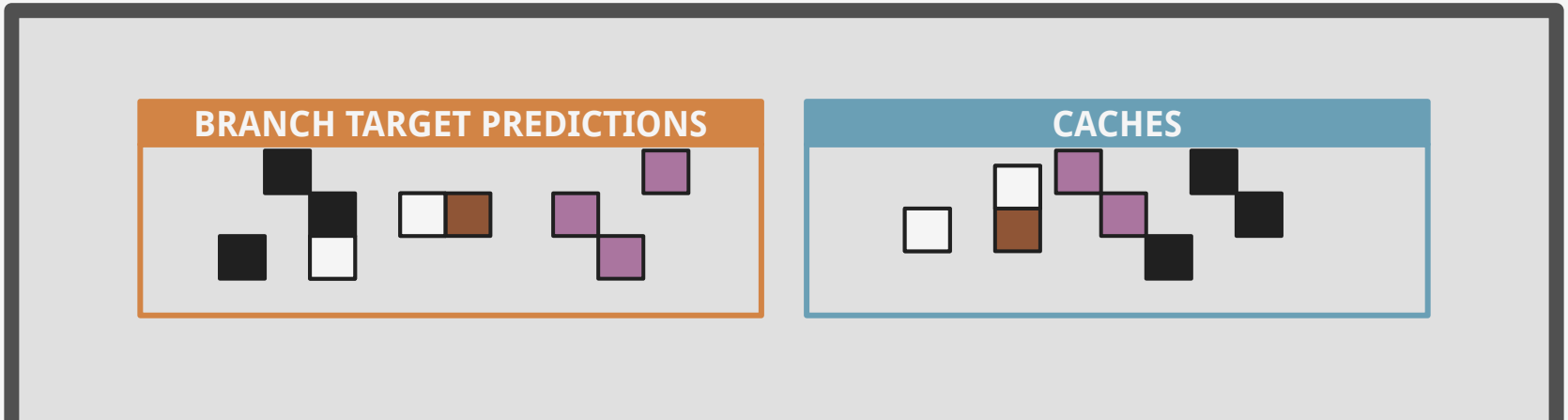
BRANCH TARGET PREDICTIONS



CACHES

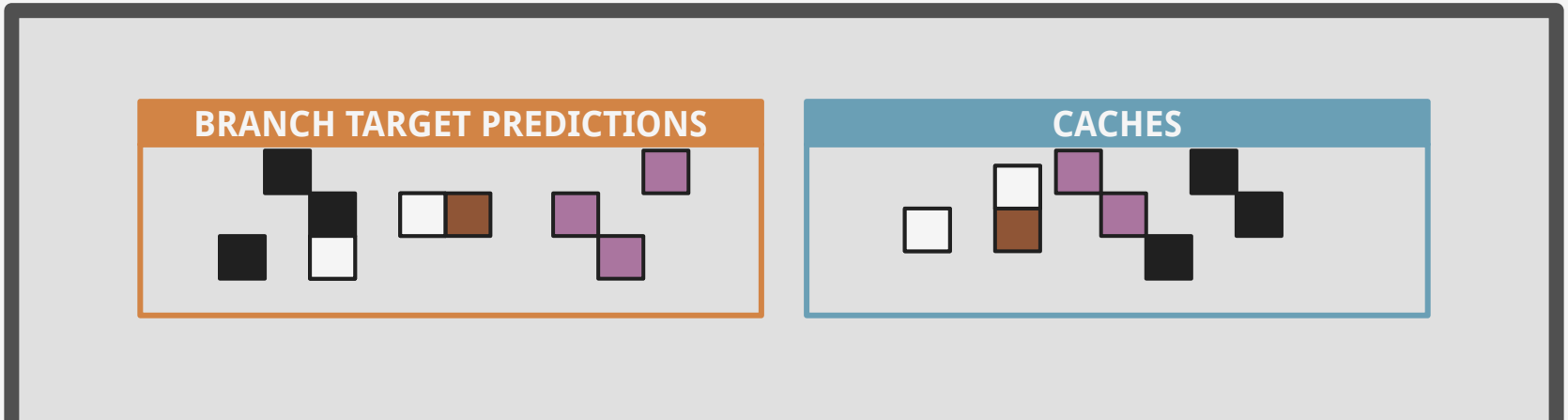
VMM (“ring -1”)

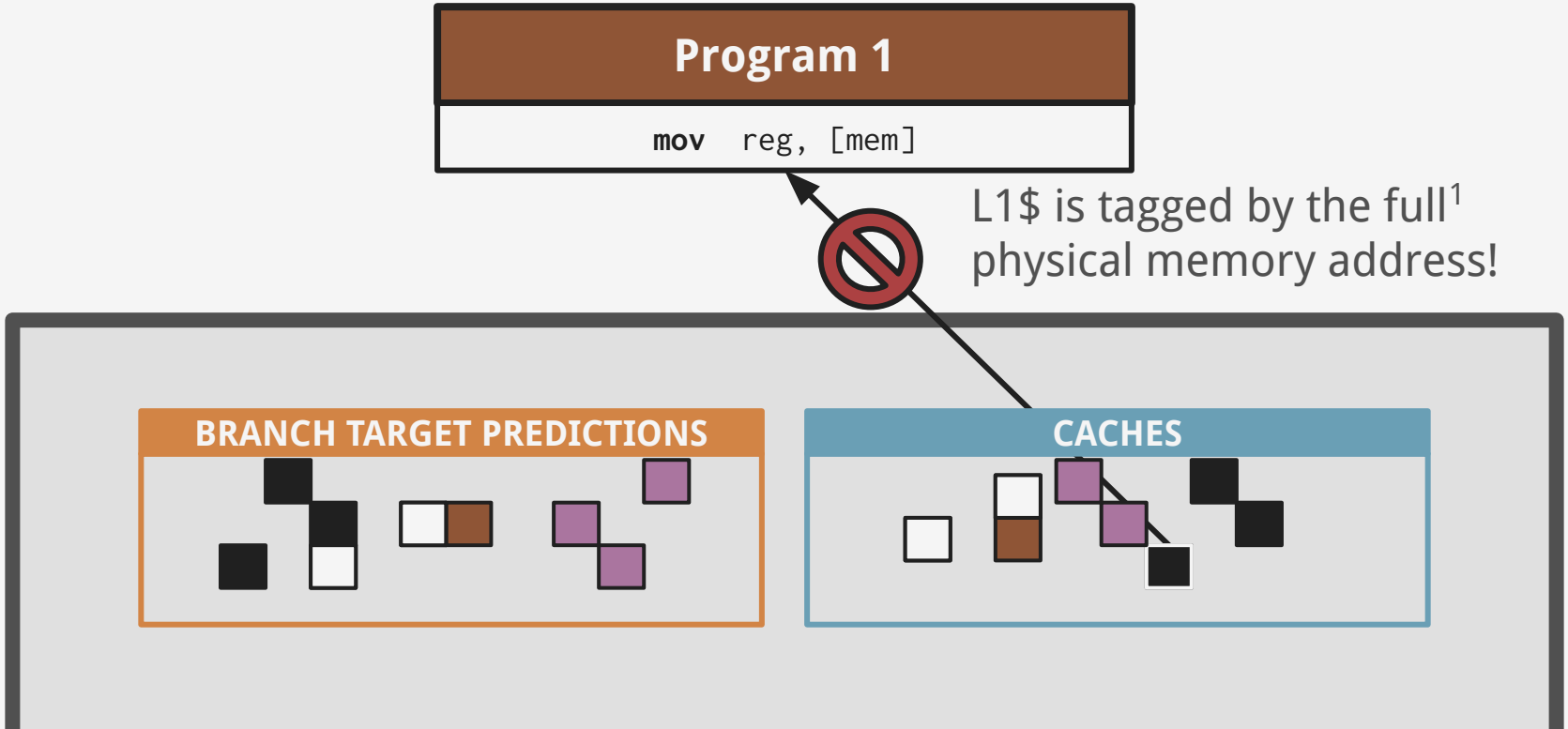
`jmp… mov… mov… jmp… etc`



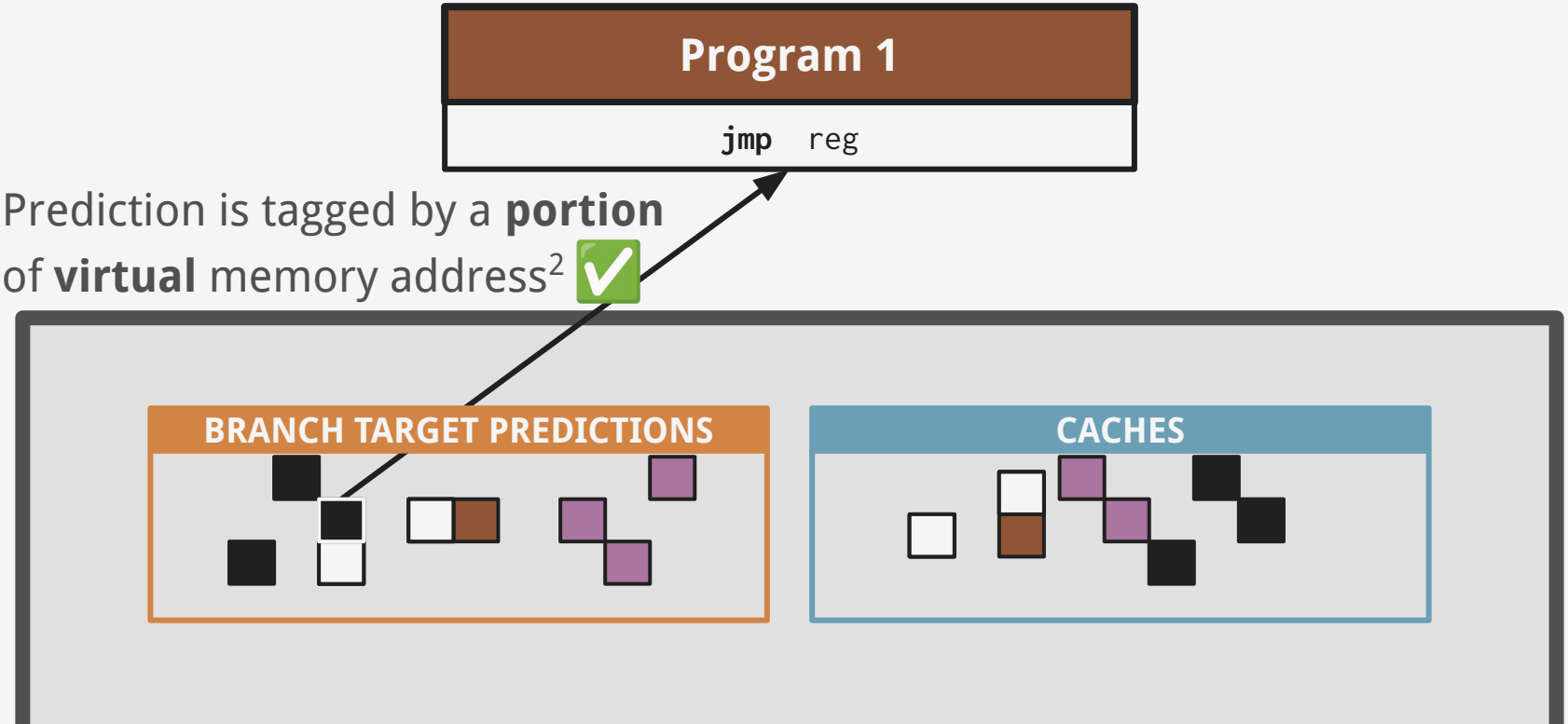
Program 1

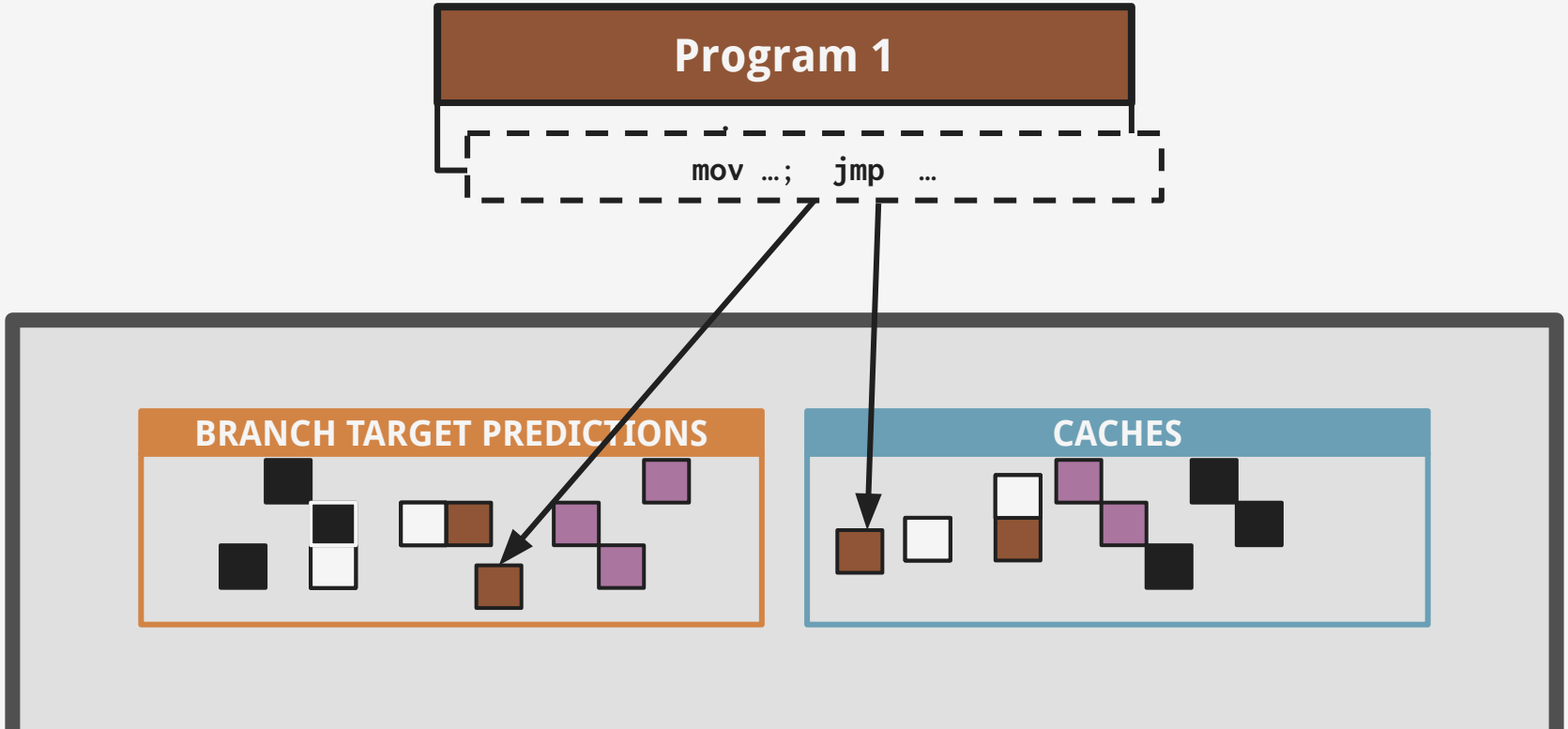
```
mov  reg, [mem]
```



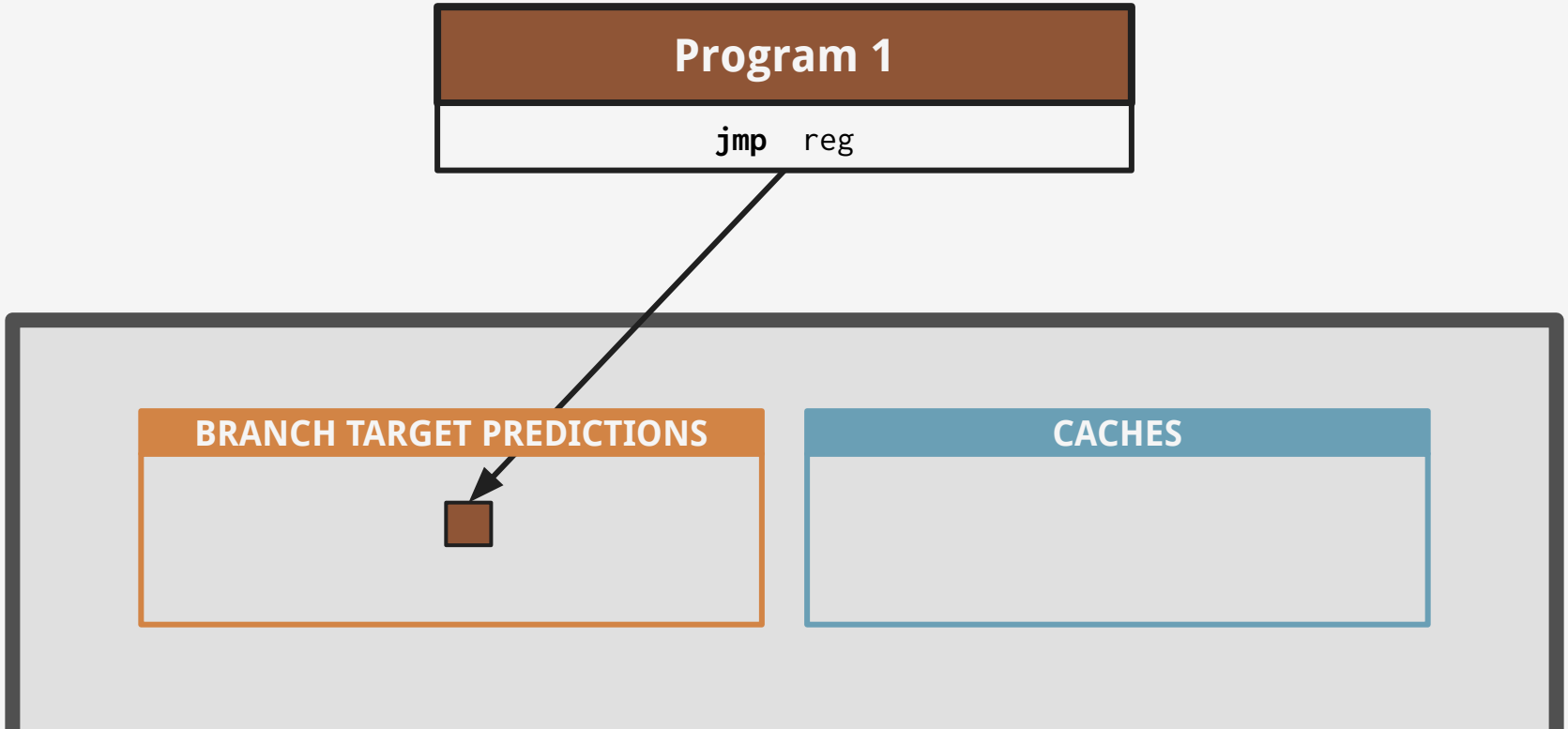

Program 1
mov reg, [mem]
L1$ is tagged by the full[1] physical memory address!
BRANCH TARGET PREDICTIONS
CACHES

👷 *So what? It's a design choice.*



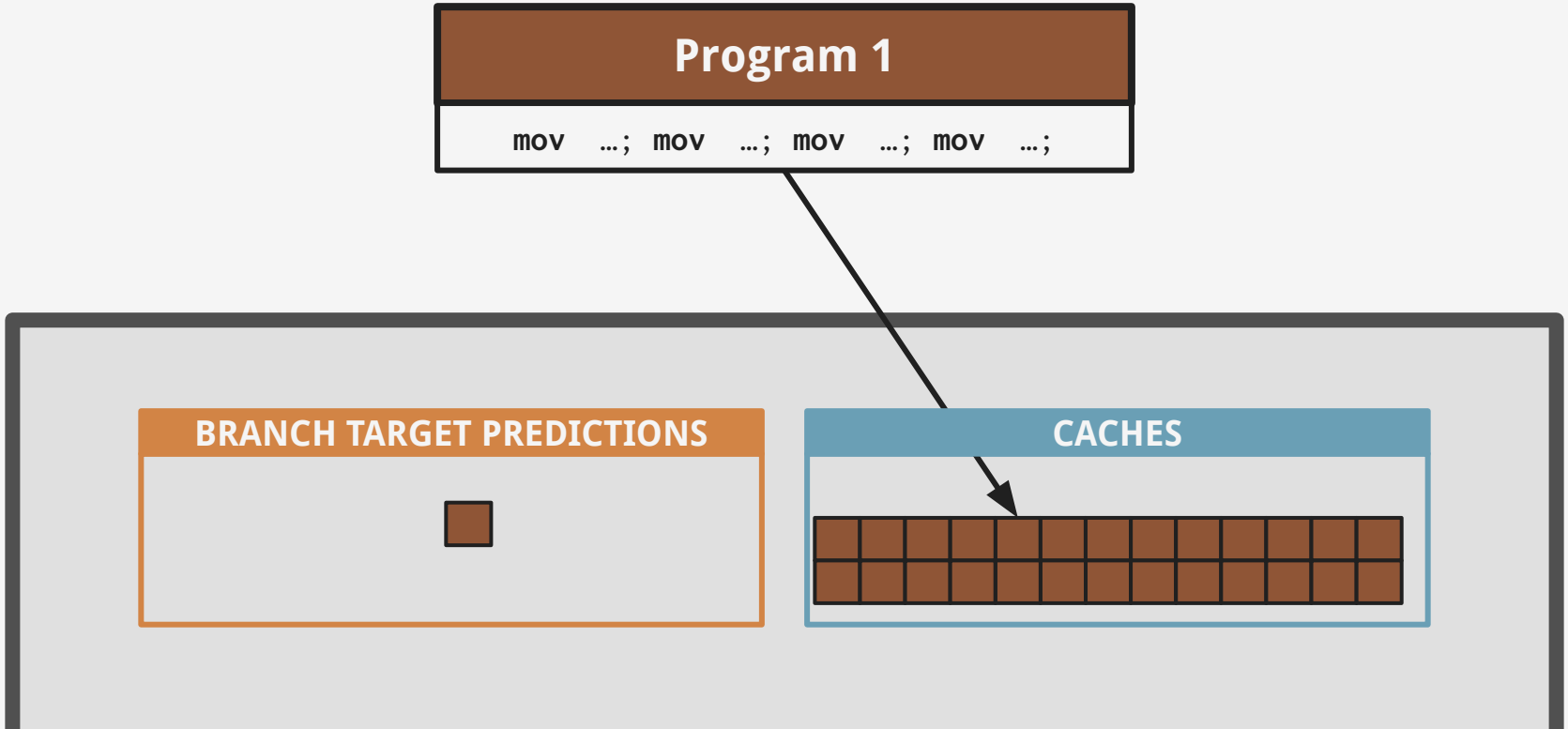

Program 1
mov …; jmp …
BRANCH TARGET PREDICTIONS
CACHES

Inject prediction



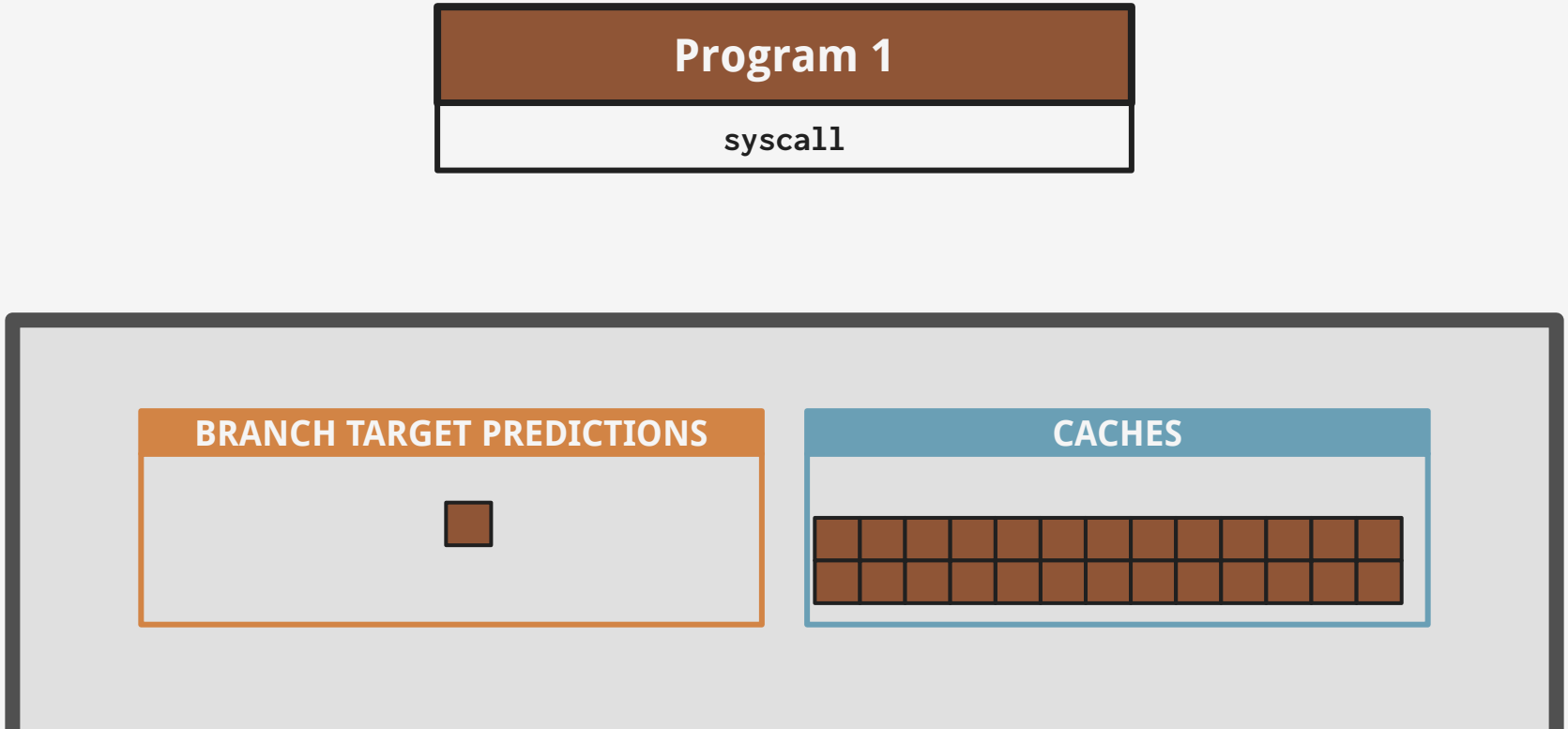
Inject prediction

Prime side channel



Inject prediction

Prime side channel



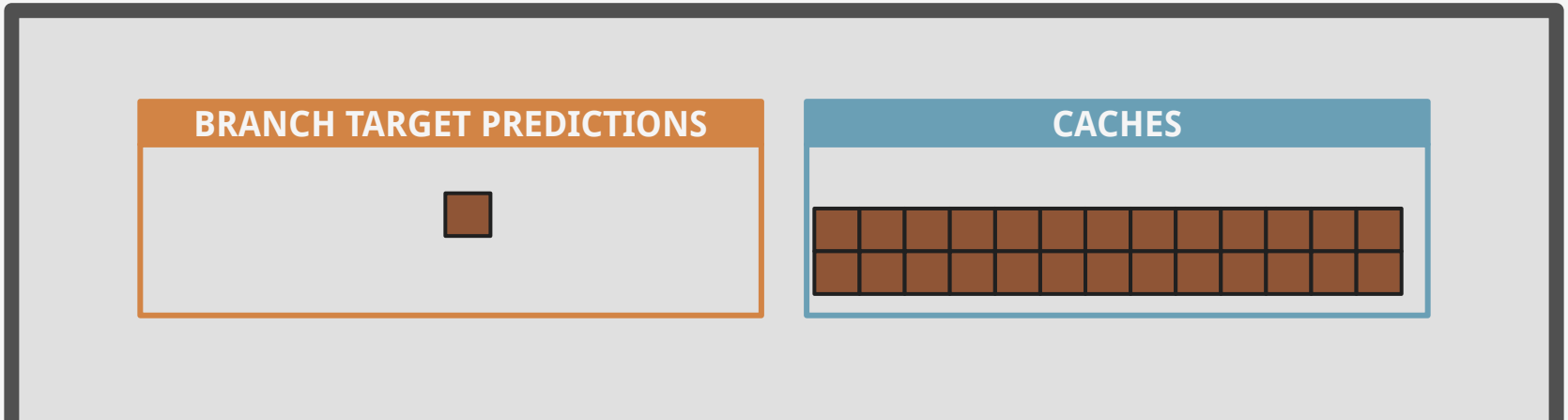
Inject prediction

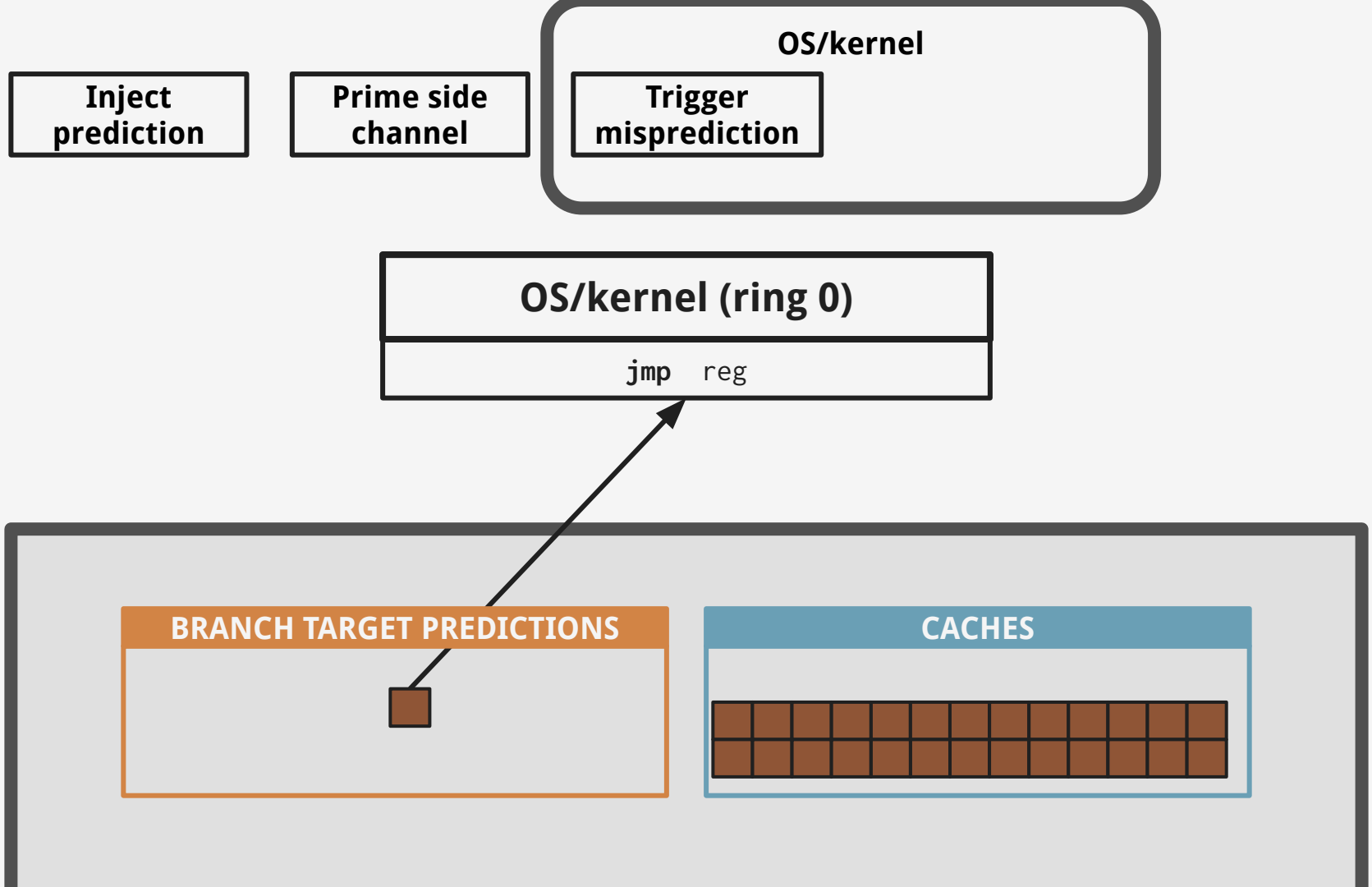
Prime side channel

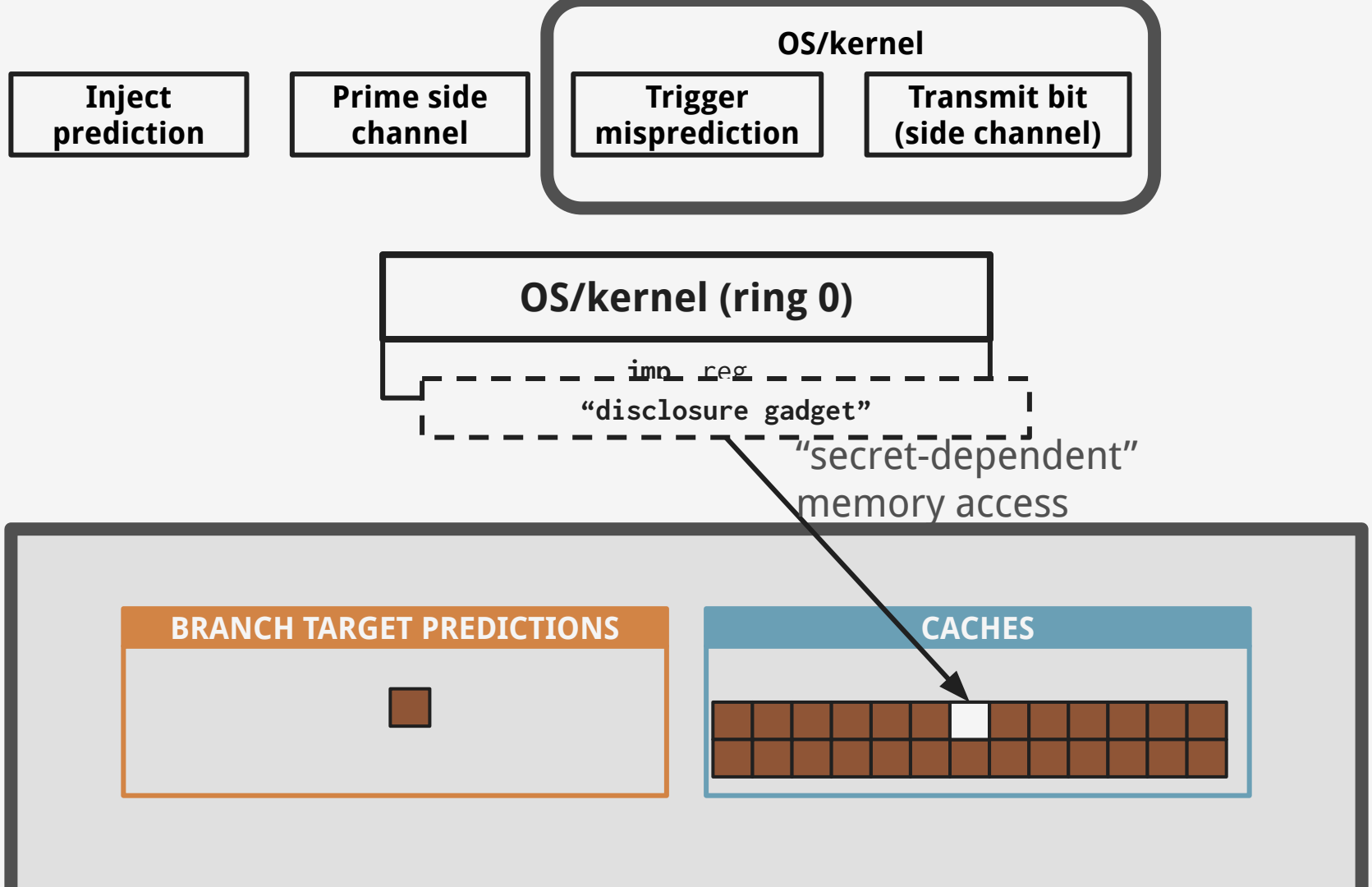
OS/kernel

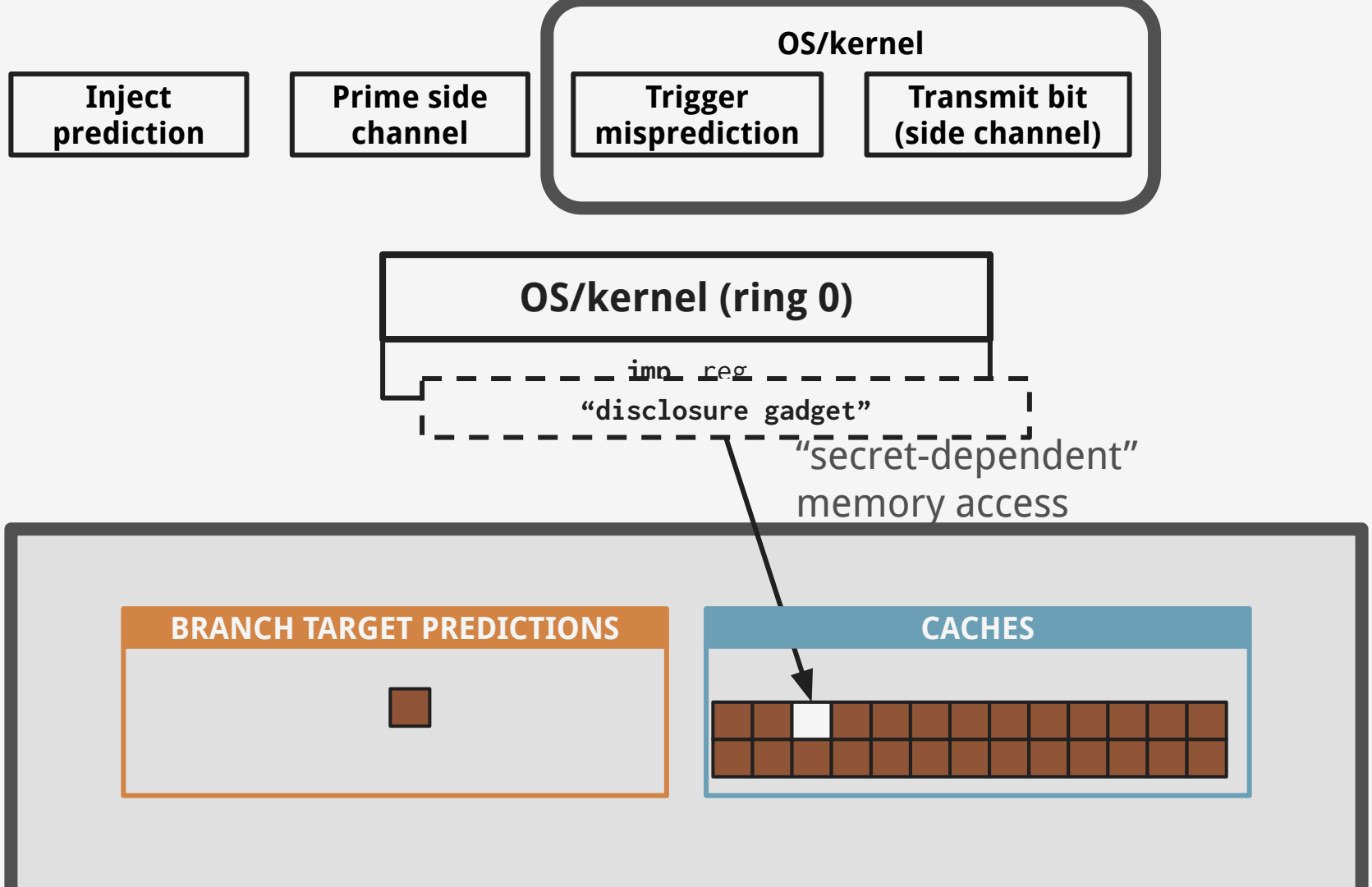
OS/kernel (ring 0)

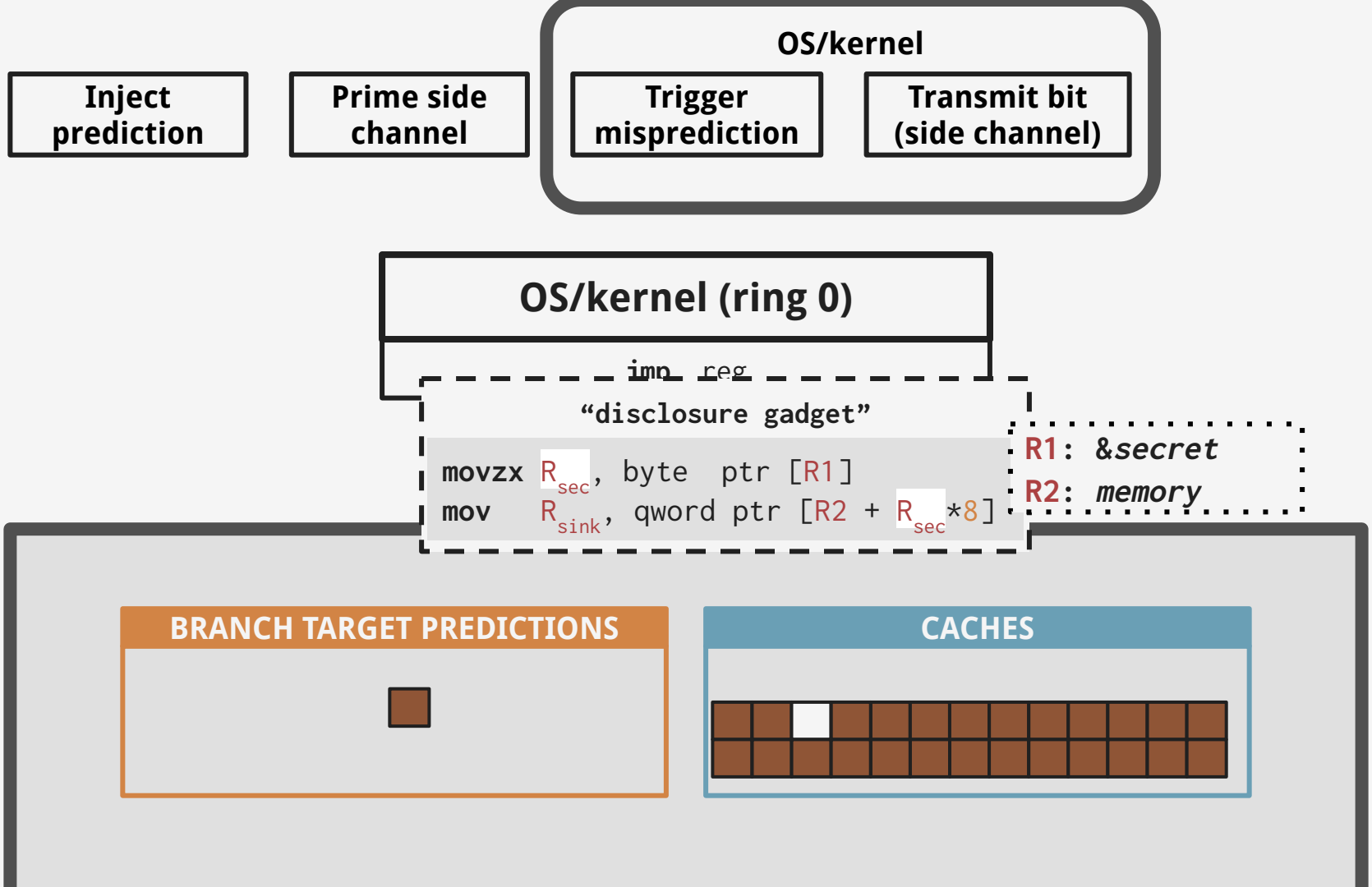
```
jmp  reg
```

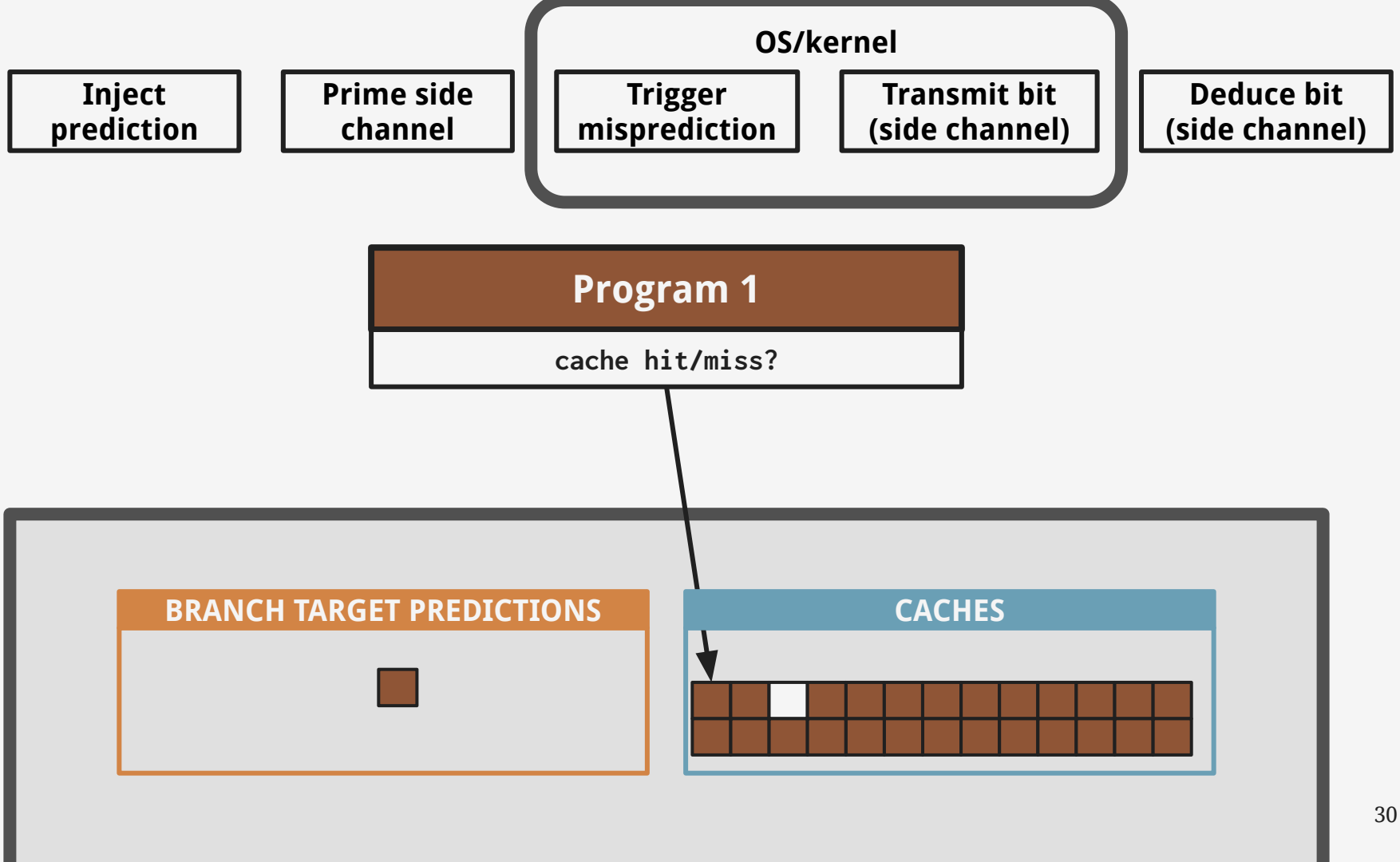


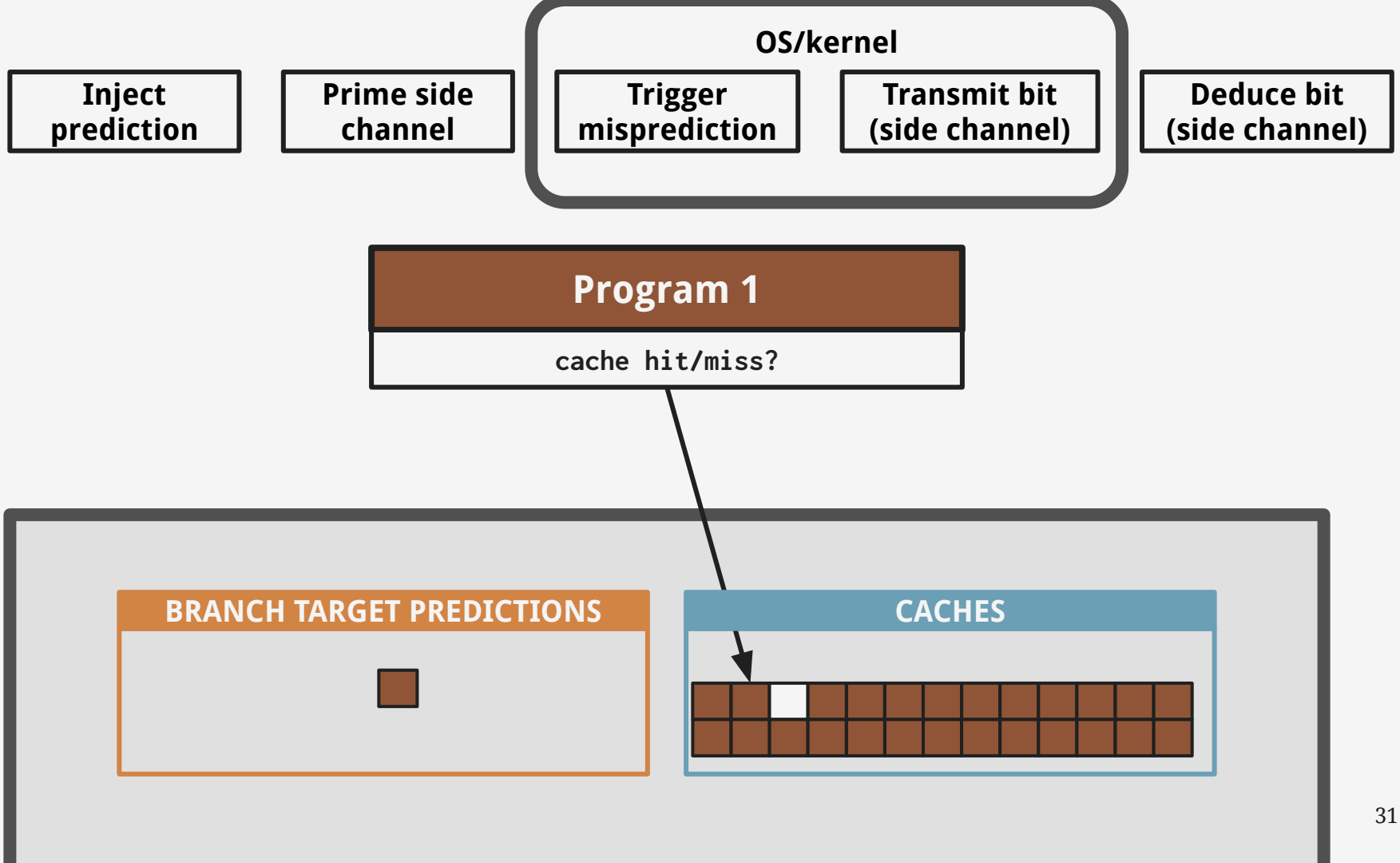

OS/kernel
Inject prediction
Prime side channel
Trigger misprediction
OS/kernel (ring 0)
jmp reg
BRANCH TARGET PREDICTIONS
CACHES

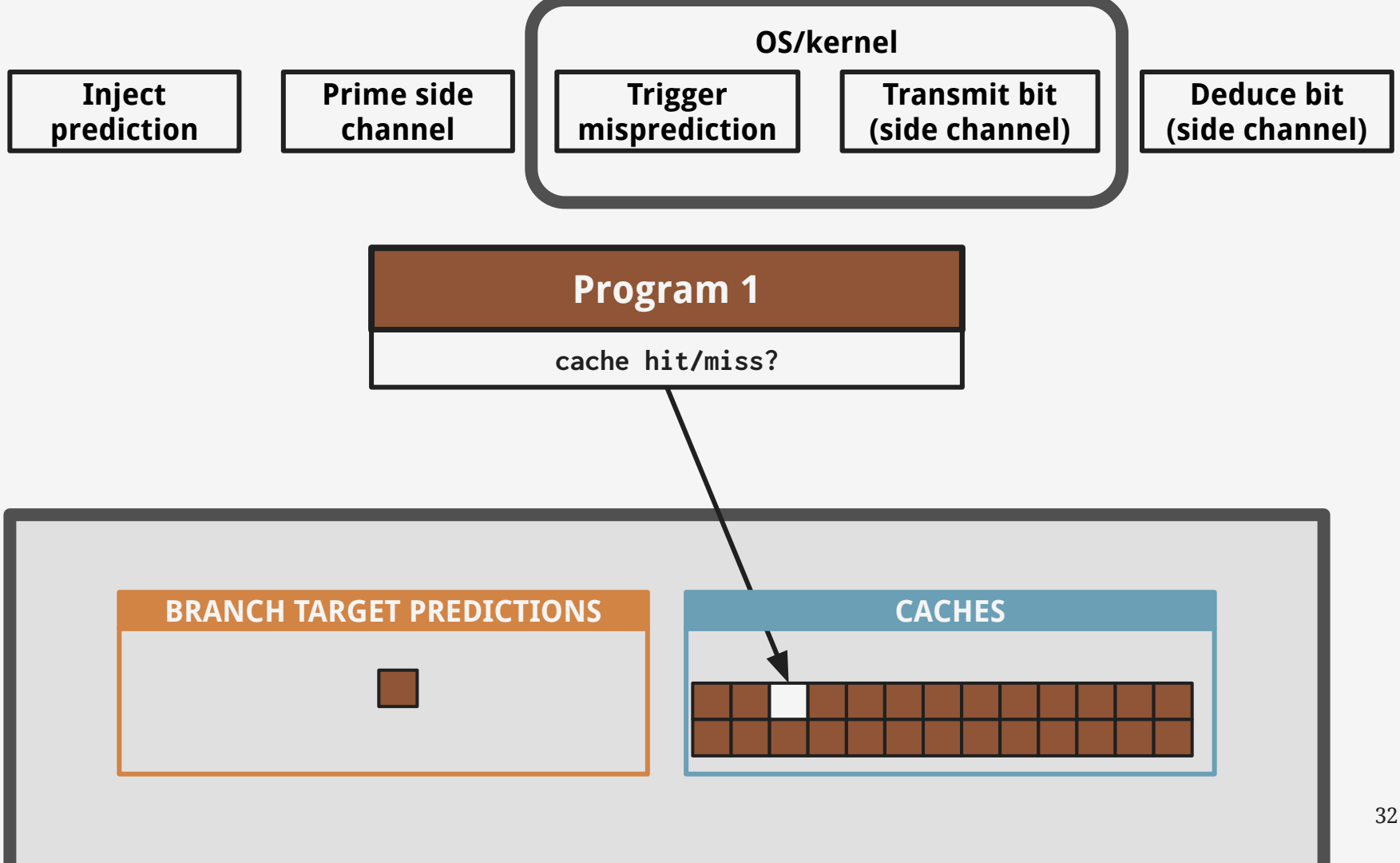

OS/kernel
Inject prediction
Prime side channel
Trigger misprediction
Transmit bit (side channel)
OS/kernel (ring 0)
jmp reg
“disclosure gadget”
“secret-dependent” memory access
BRANCH TARGET PREDICTIONS
CACHES

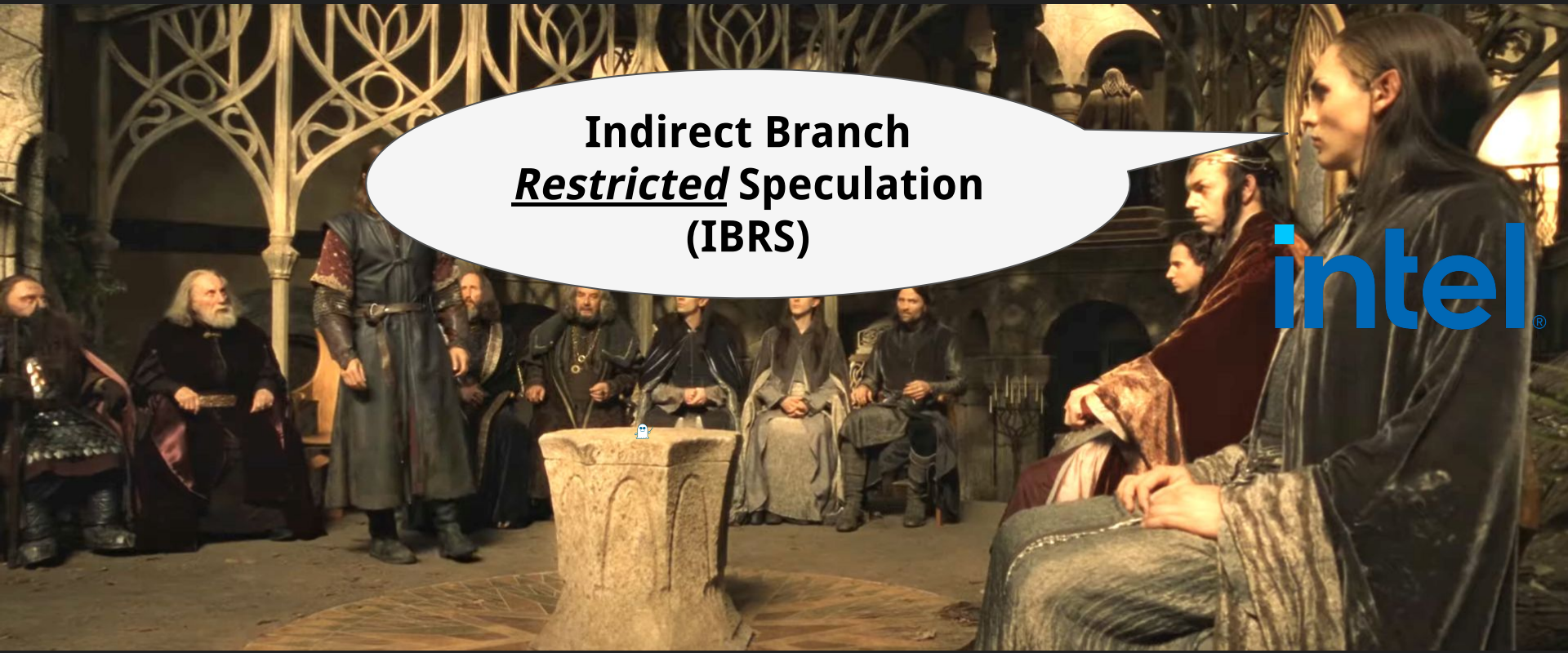

Inject prediction
Prime side channel
OS/kernel
Trigger misprediction
Transmit bit (side channel)
OS/kernel (ring 0)
jmp reg
“disclosure gadget”
“secret-dependent” memory access
BRANCH TARGET PREDICTIONS
CACHES

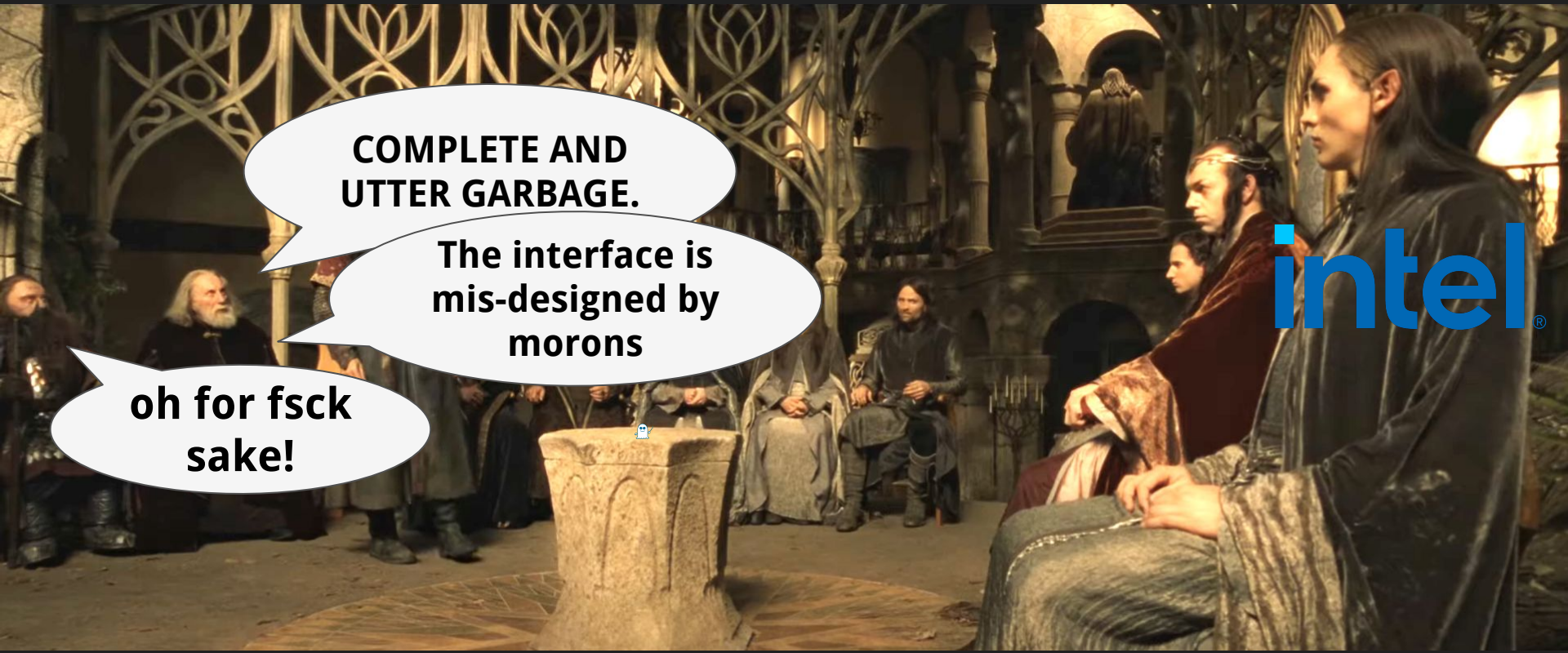

Inject prediction
Prime side channel
OS/kernel
Trigger misprediction
Transmit bit (side channel)
OS/kernel (ring 0)
jmp reg
“disclosure gadget”
movzx R_sec, byte ptr [R1]
mov R_sink, qword ptr [R2 + R_sec*8]
R1: &secret
R2: memory
BRANCH TARGET PREDICTIONS
CACHES

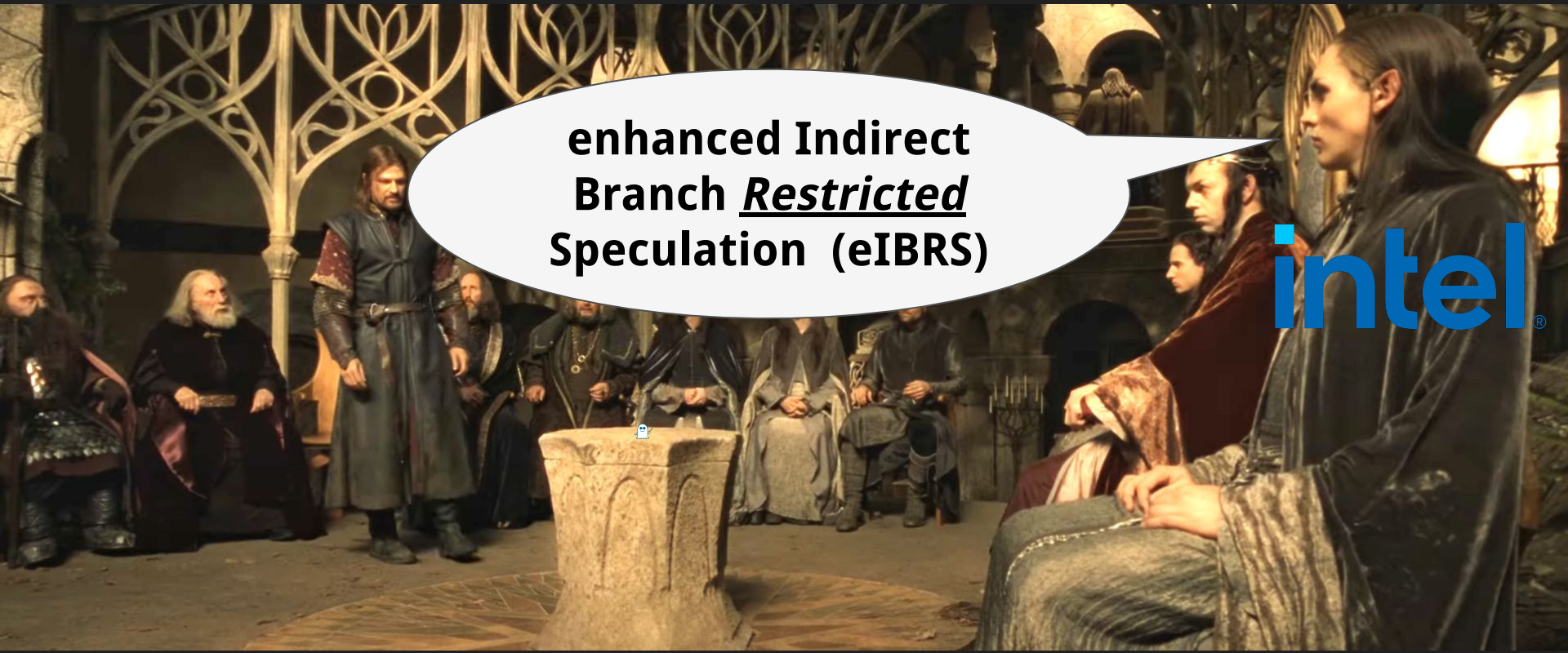

OS/kernel
Inject prediction
Prime side channel
Trigger misprediction
Transmit bit (side channel)
Deduce bit (side channel)
Program 1
cache hit/miss?
BRANCH TARGET PREDICTIONS
CACHES


OS/kernel
Inject prediction
Prime side channel
Trigger misprediction
Transmit bit (side channel)
Deduce bit (side channel)
Program 1
cache hit/miss?
BRANCH TARGET PREDICTIONS
CACHES


Inject prediction
Prime side channel
OS/kernel
Trigger misprediction
Transmit bit (side channel)
Deduce bit (side channel)
Program 1
cache hit/miss?
BRANCH TARGET PREDICTIONS
CACHES


Indirect Branch
Restricted Speculation
(IBRS)
intel


COMPLETE AND UTTER GARBAGE.
The interface is mis-designed by morons
oh for fsck sake!
intel


enhanced Indirect
Branch *Restricted*
Speculation (eIBRS)
intel

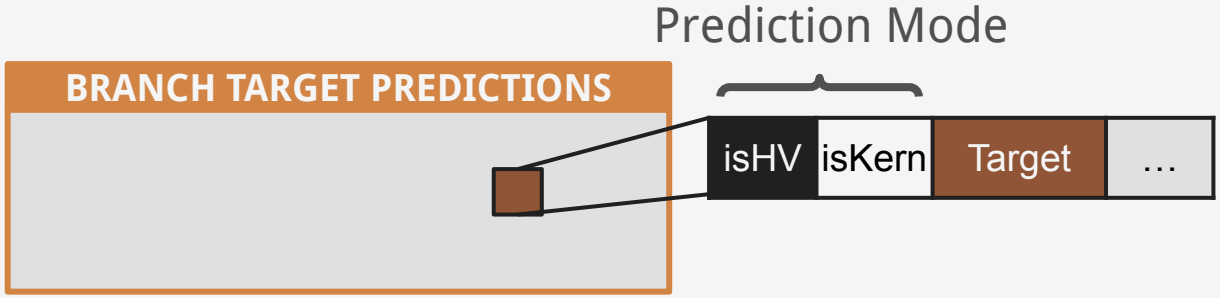

Love the vibe in the room rn
Great meeting everyone
Great job team
I think we made some real progress

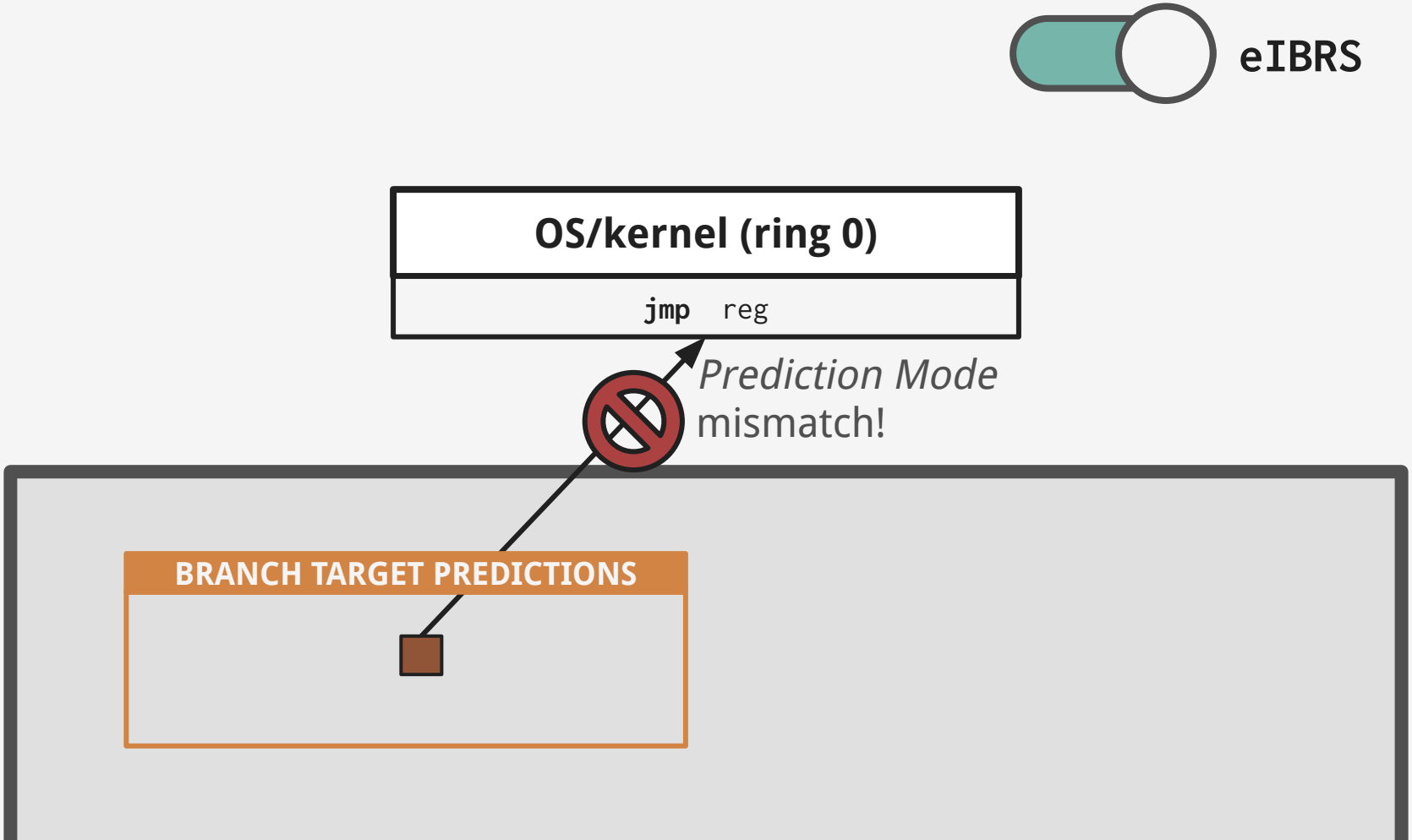
eIBRS

- `asm("wrmsr" :: "c"(SPEC_CTRL), "a"(1), "d"(0));`

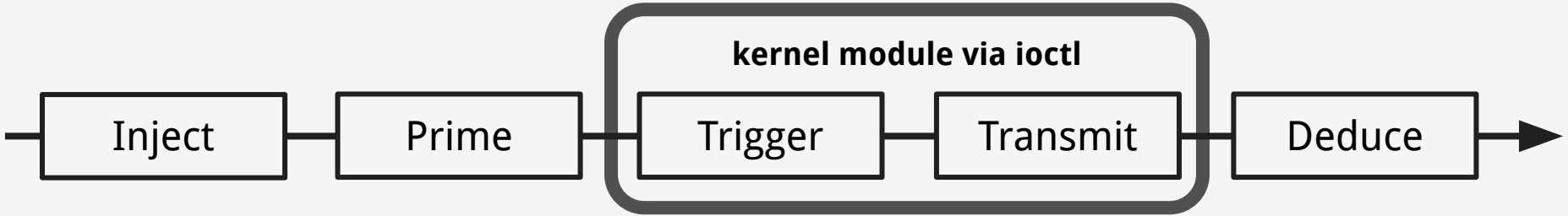
eIBRS

- `asm("wrmsr" :: "c"(SPEC_CTRL), "a"(1), "d"(0));`
- set and forget

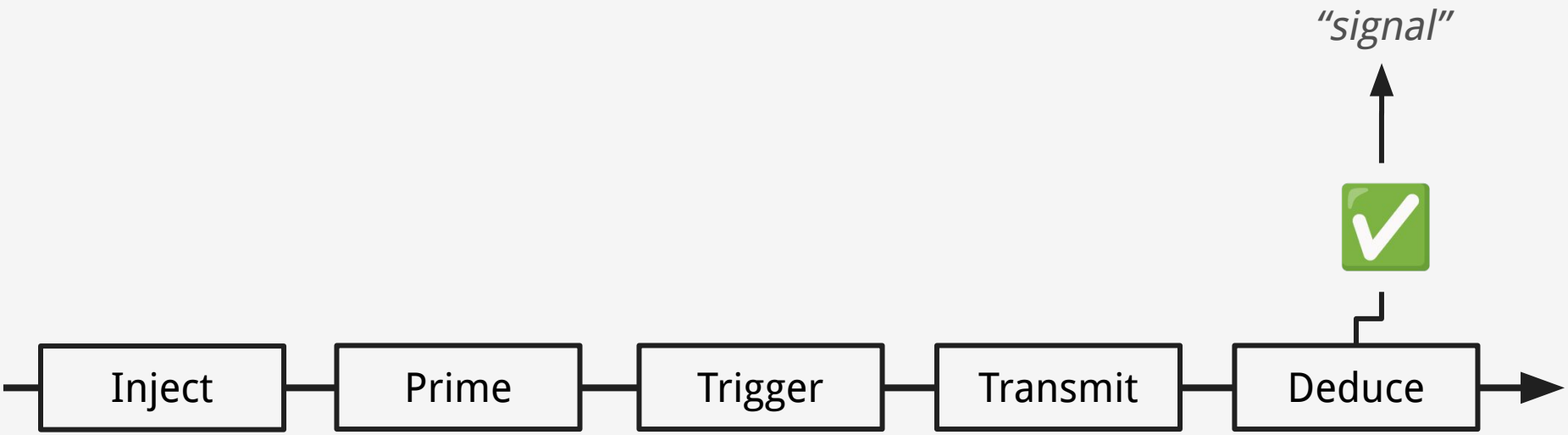



eIBRS
OS/kernel (ring 0)
jmp reg
Prediction Mode
mismatch!
BRANCH TARGET PREDICTIONS

Assessing eIBRS

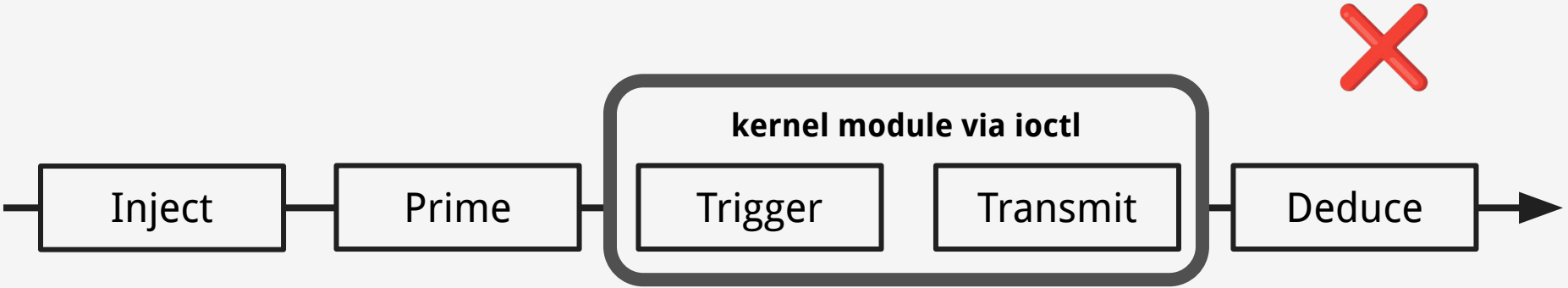


Assessing eIBRS



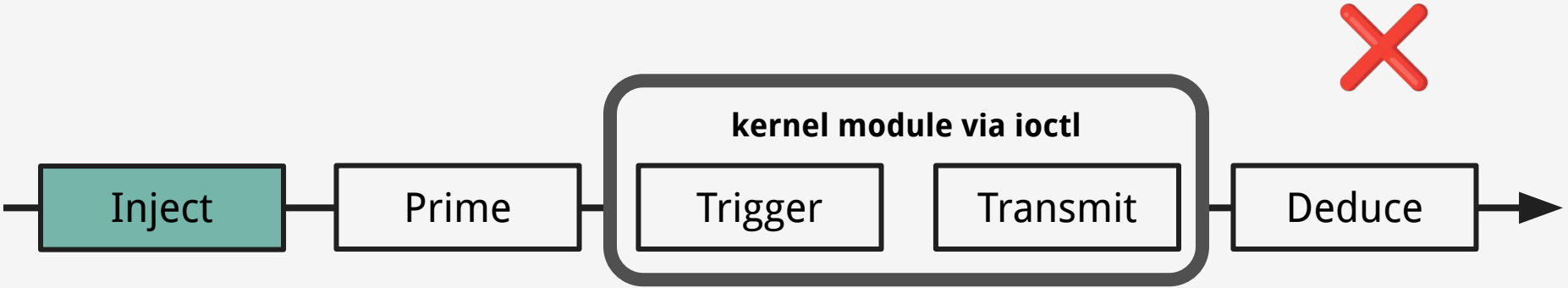
Assessing eIBRS

- Is eIBRS secretly turned on by firmware?
- **Inconclusive:** Does it really fail because of eIBRS?

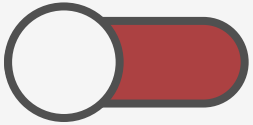


Assessing eIBRS

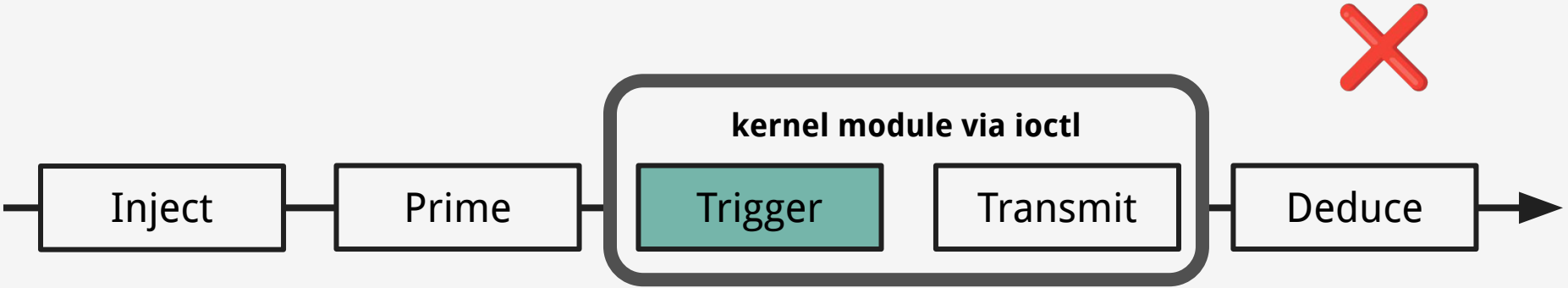
- Is eIBRS secretly turned on by firmware?
- **Inconclusive:** Does it really fail because of eIBRS?



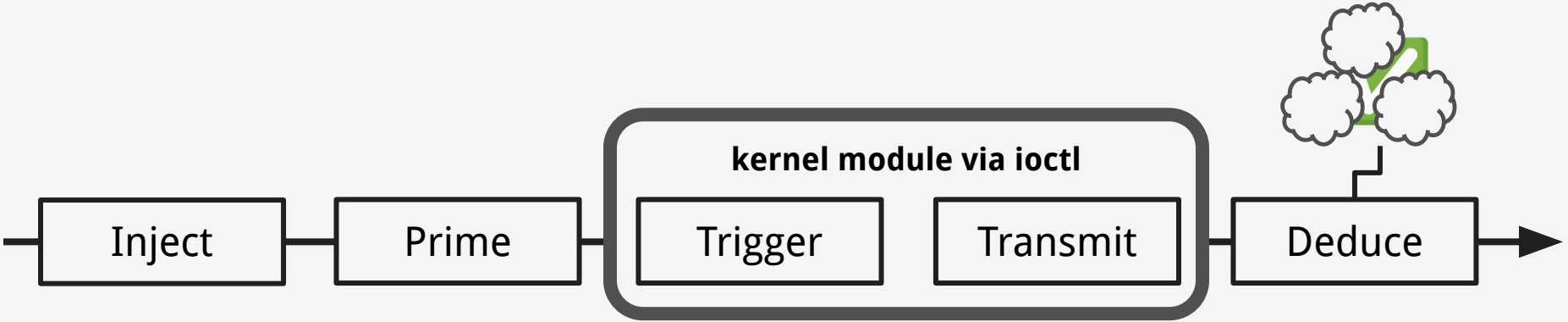
Assessing eIBRS



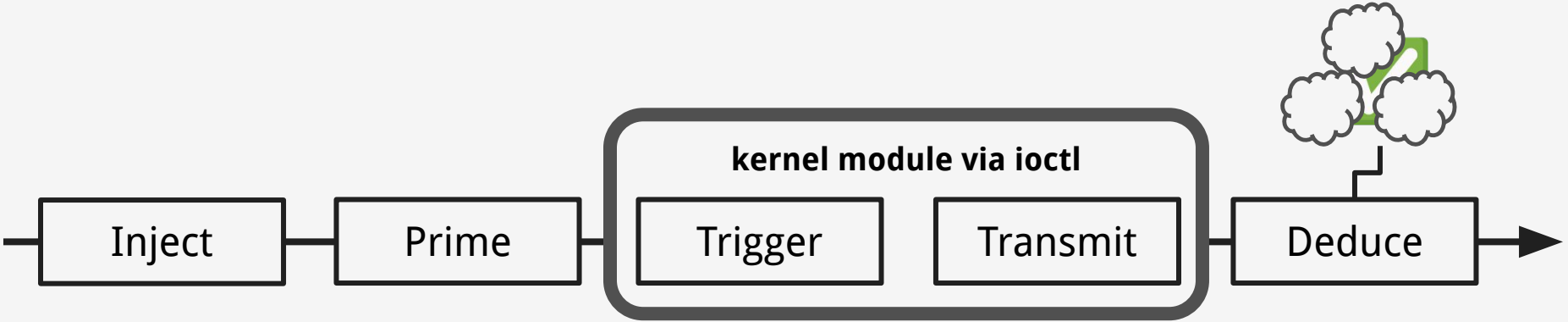
- Is eIBRS secretly turned on by firmware?
- **Inconclusive:** Does it really fail because of eIBRS?



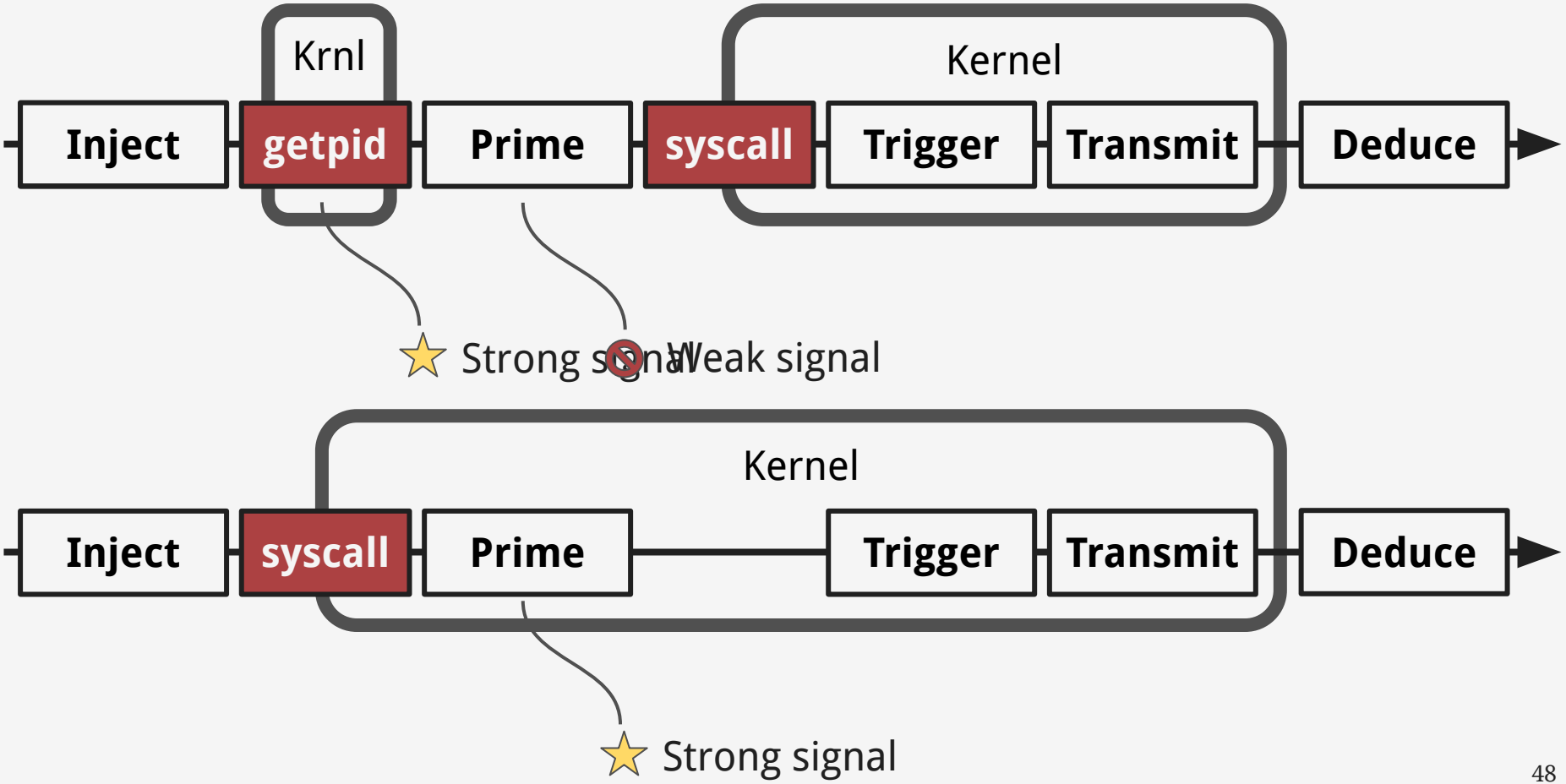
Alder Lake, Golden Cove

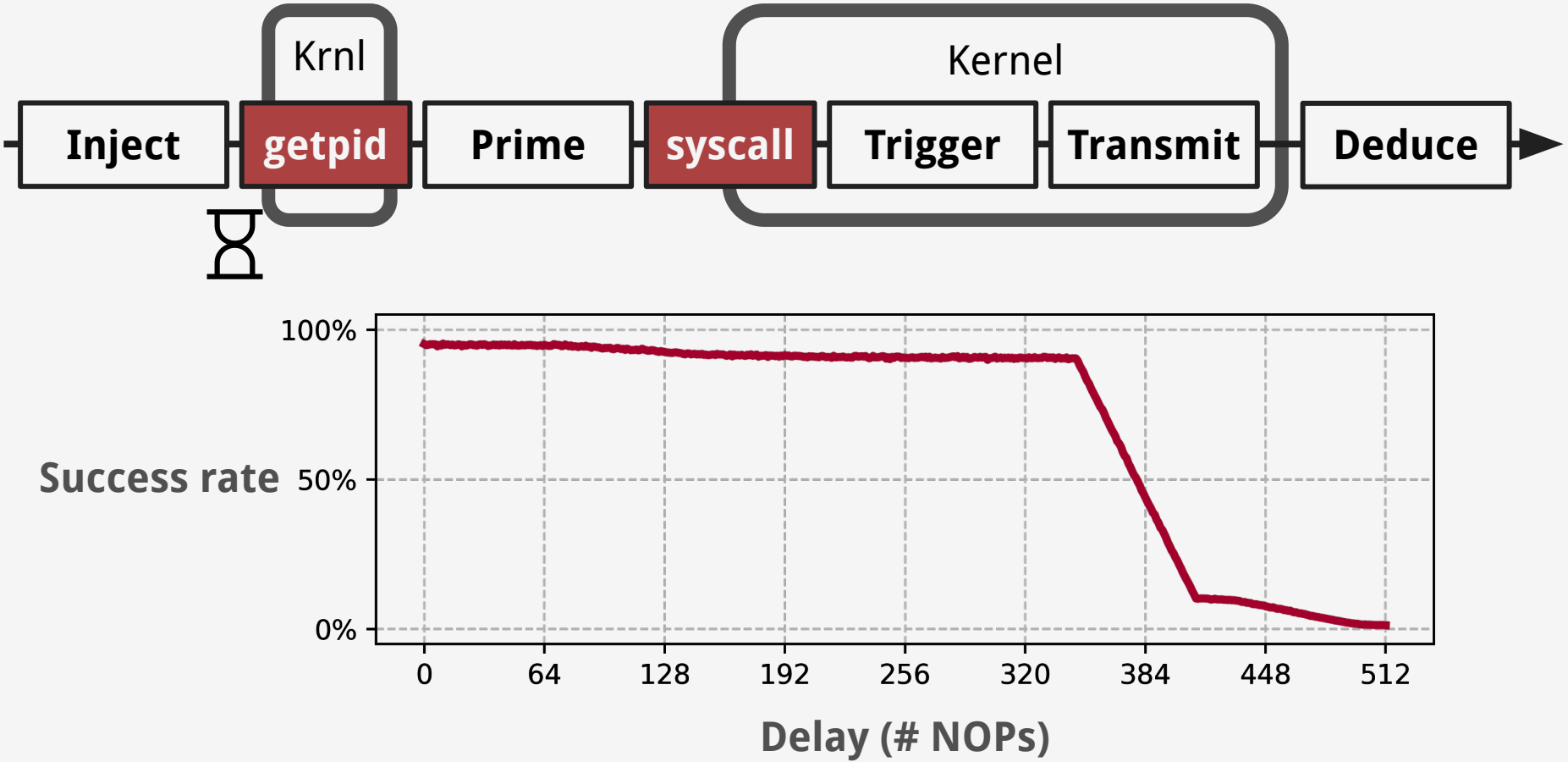


Alder Lake, Golden Cove



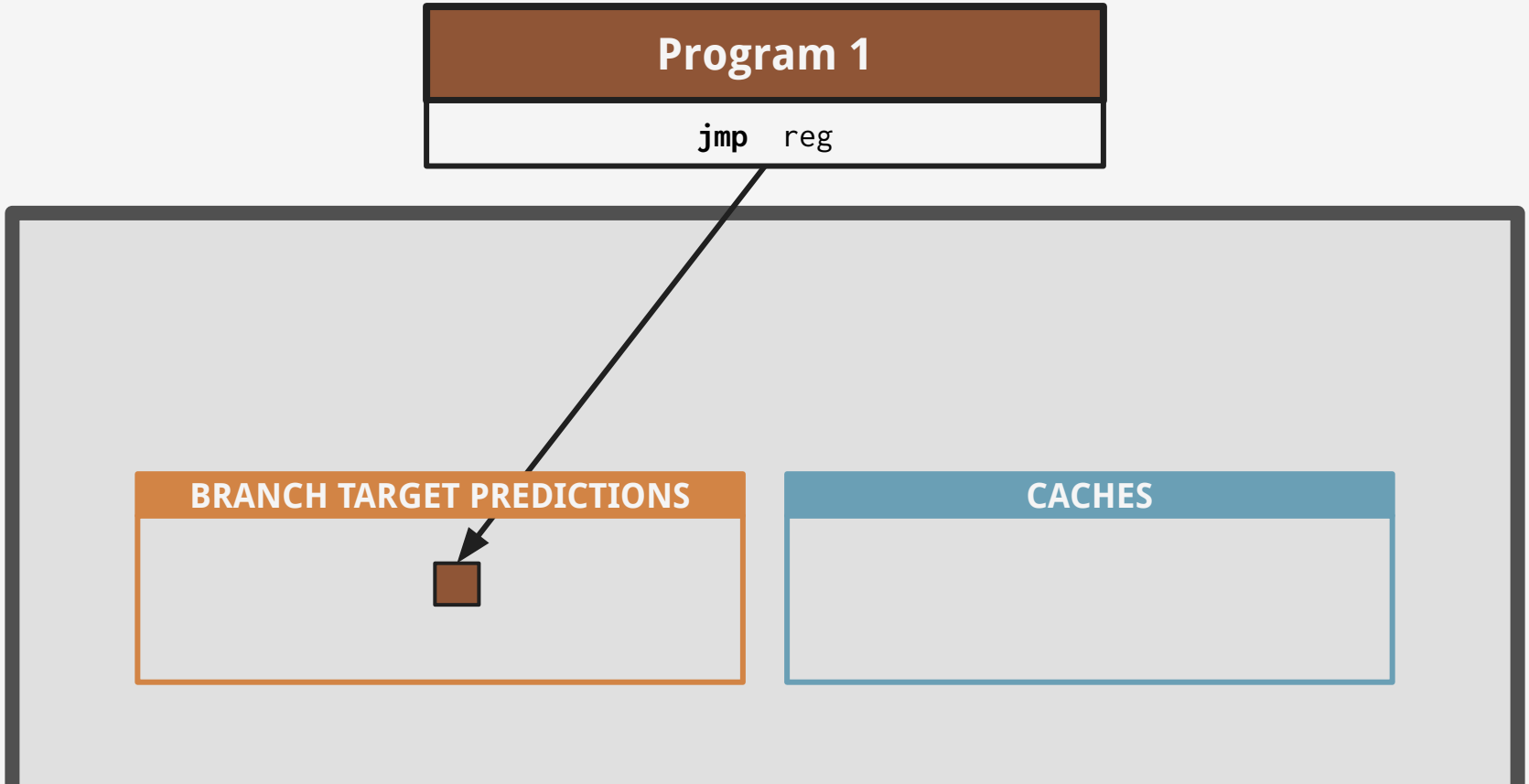
What is going on?


Krnl
Kernel
Inject
getpid
Prime
syscall
Trigger
Transmit
Deduce
Strong signal
Weak signal
Kernel
Inject
syscall
Prime
Trigger
Transmit
Deduce
Strong signal

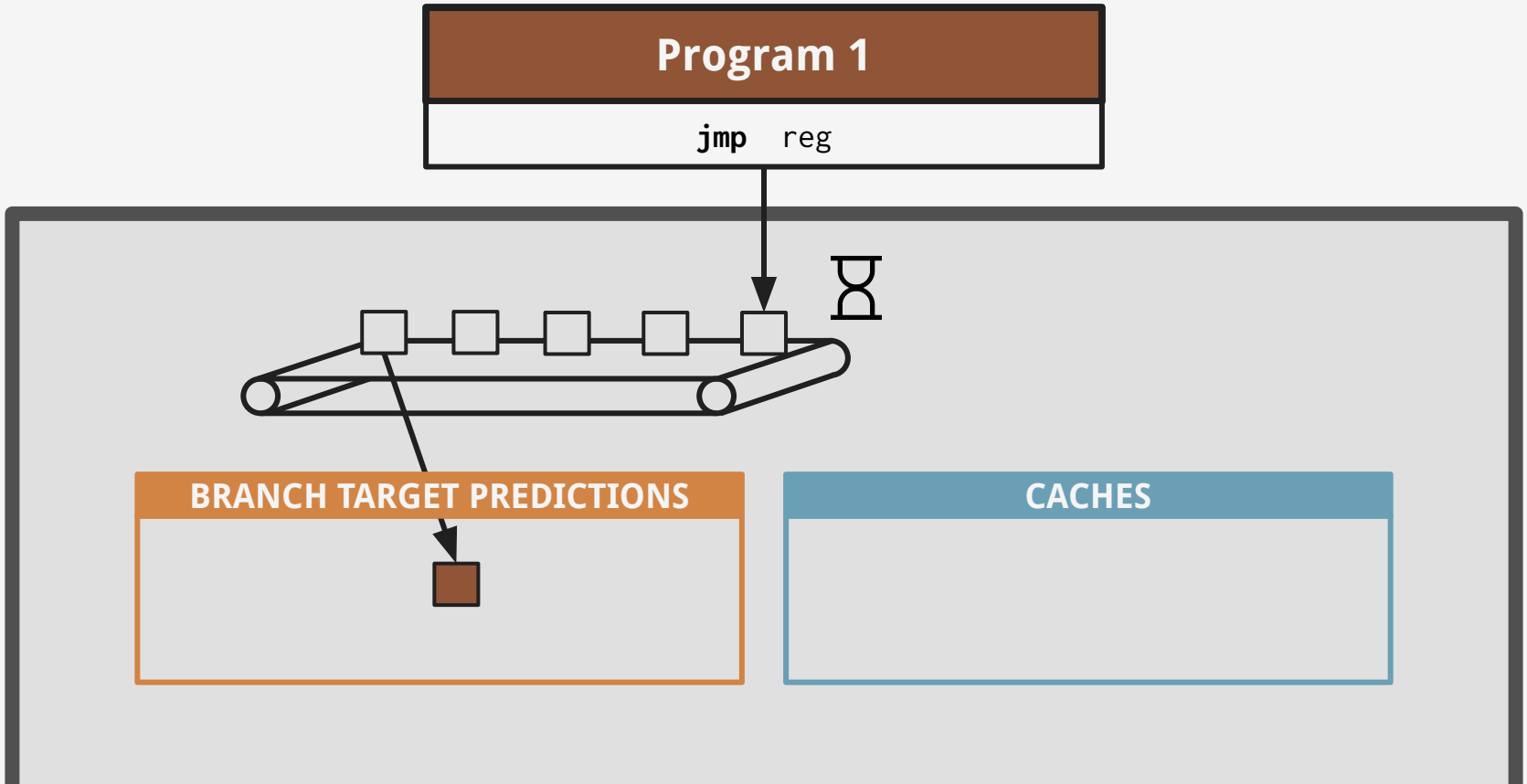

Krnl
Kernel
Inject
getpid
Prime
syscall
Trigger
Transmit
Deduce
100%
50%
0%
Success rate
0
64
128
192
256
320
384
448
512
Delay (# NOPs)

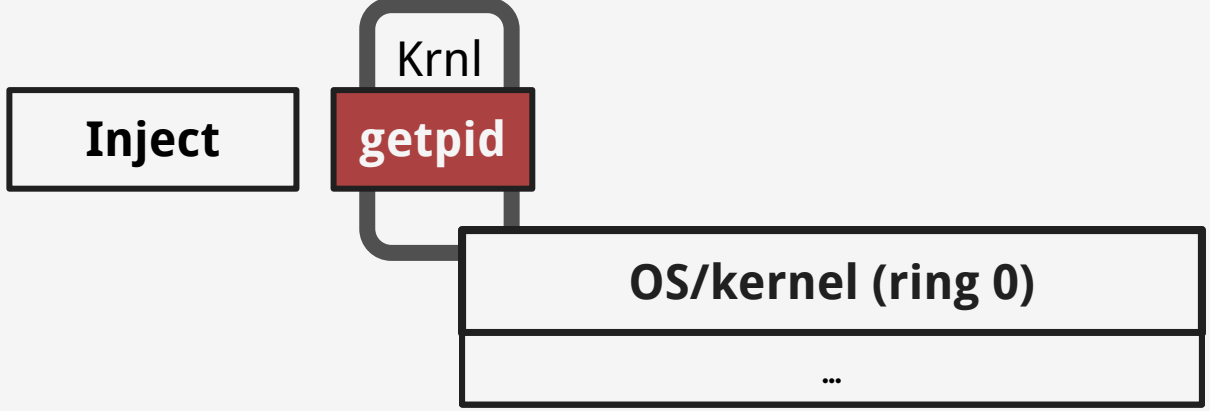
What could cause this?

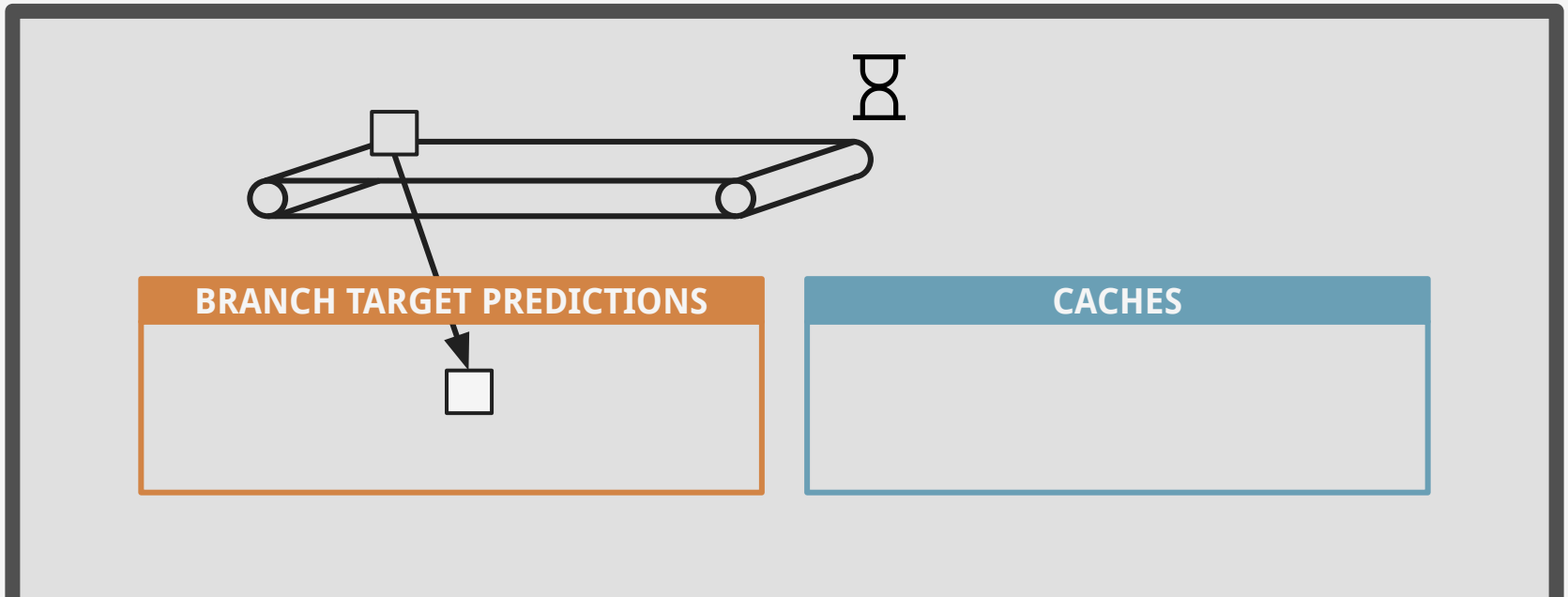
Inject



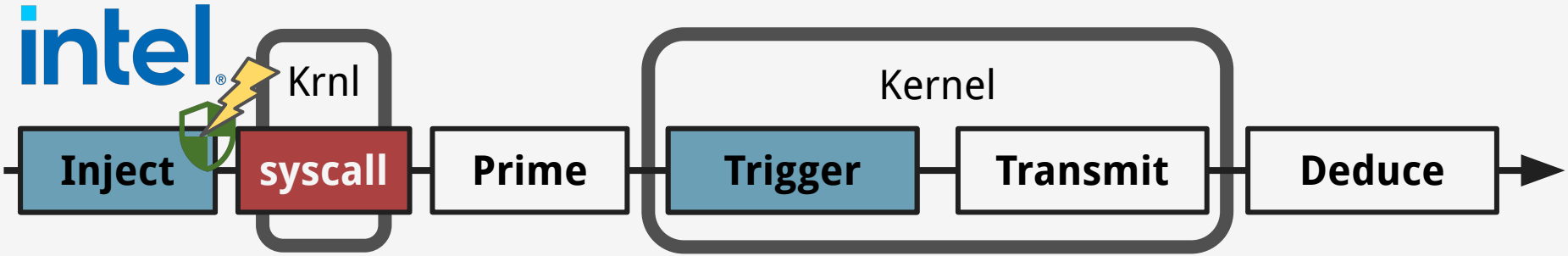
Inject

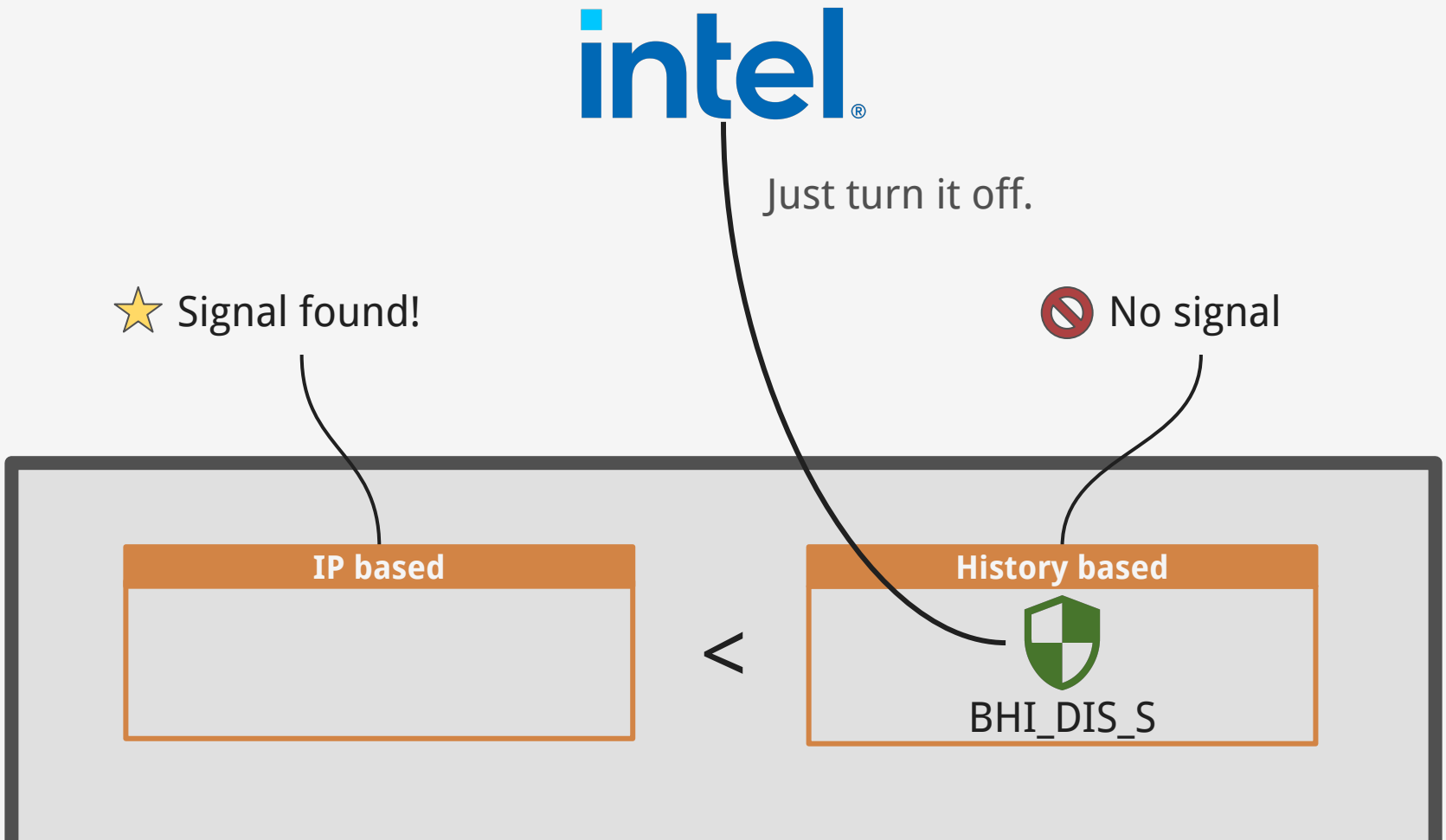



Krnl
Inject
getpid
OS/kernel (ring 0)
...

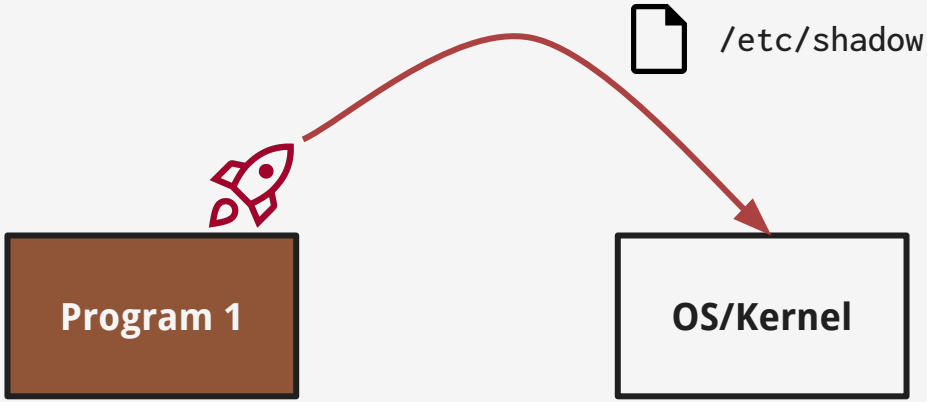

BRANCH TARGET PREDICTIONS
CACHES

Is it this easy?

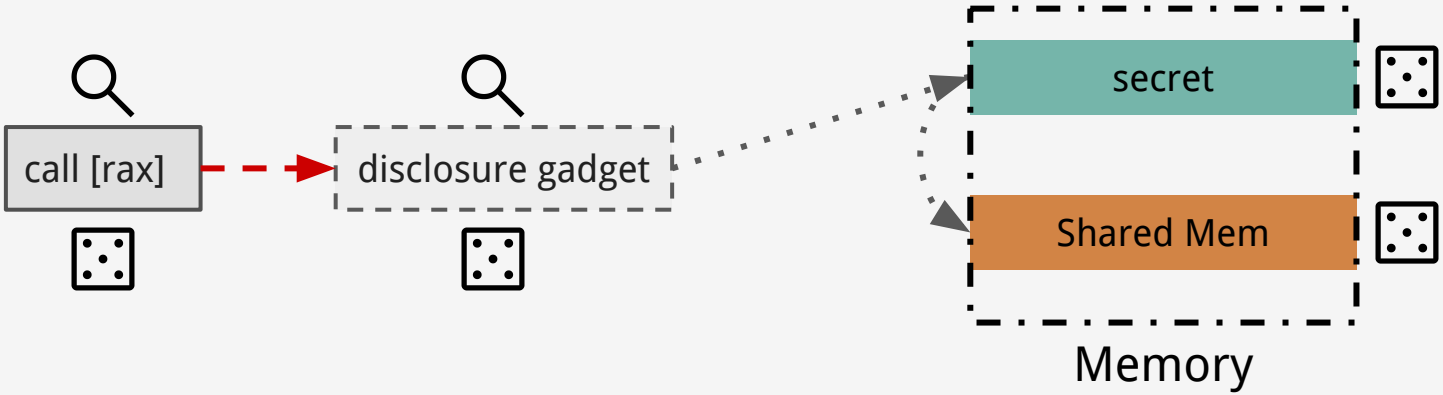

intel
Krnl
Kernel
Inject
syscall
Prime
Trigger
Transmit
Deduce

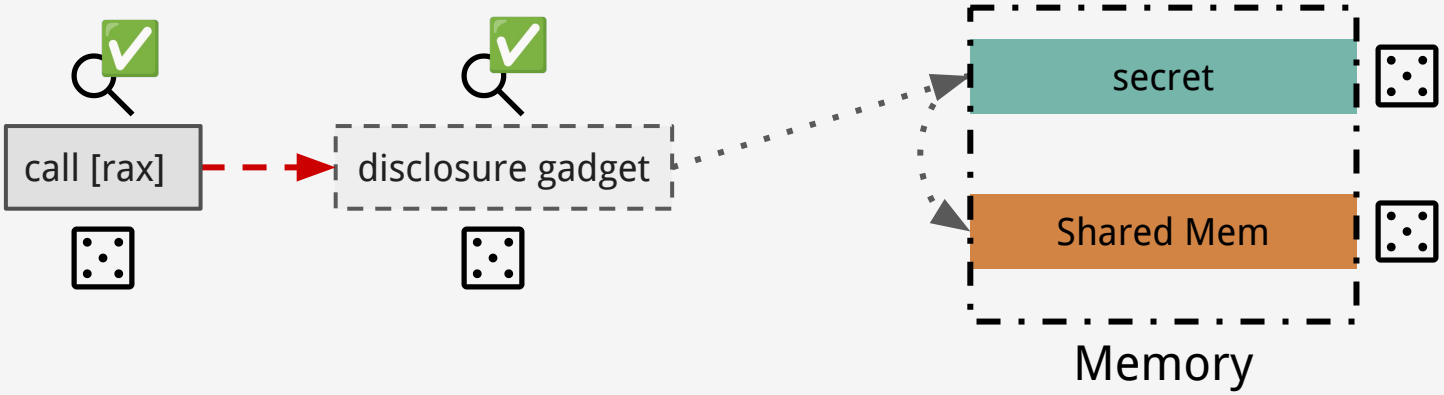

intel
Just turn it off.
Signal found!
No signal
IP based
History based
<
BHI_DIS_S

Let’s build an attack!



- Local code execution
- Know kernel image


call [rax]
disclosure gadget
secret
Shared Mem
Memory



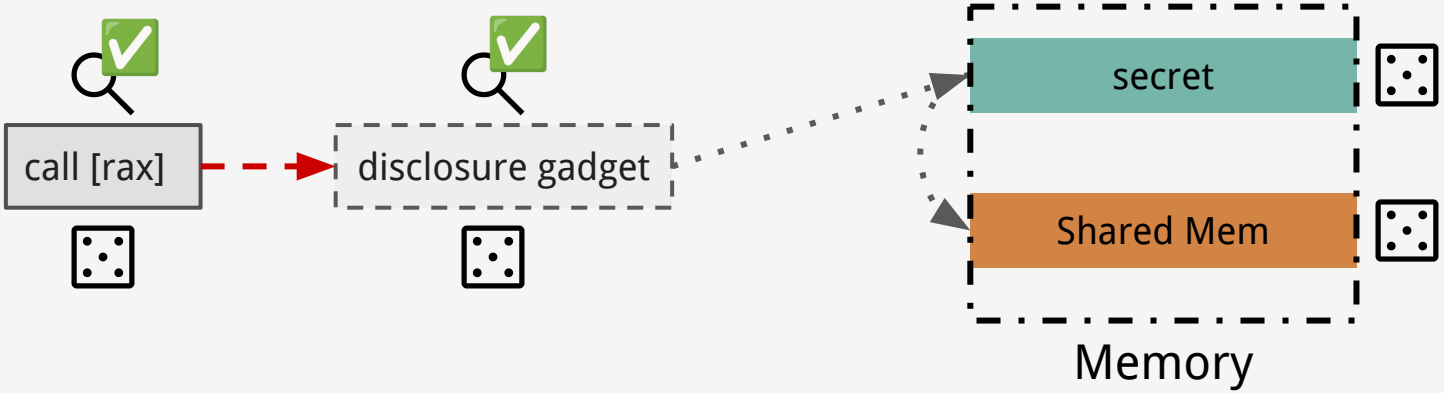
Analyze Kernel

```
key->type->read(key, buffer, buflen);
```

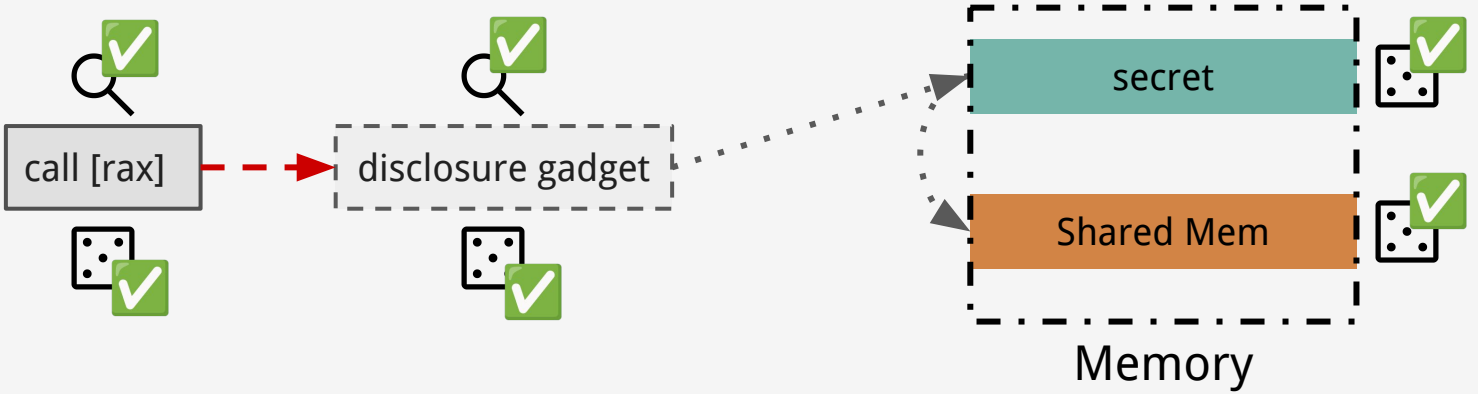
```
call [rcx]
```

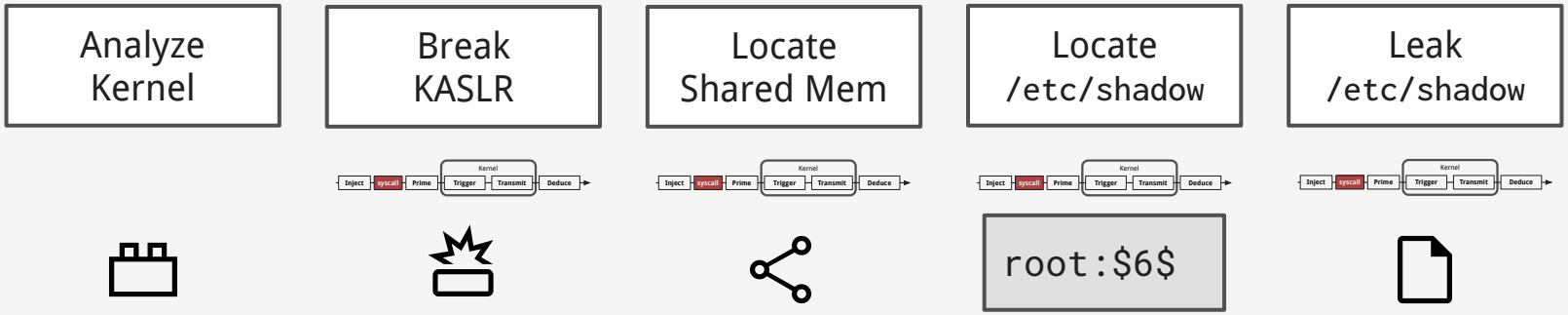
```
secret = *(uint8_t *) buffer;
*(uint64_t *) (buflen + 8 * secret);
```

```
movzx edx, byte  ptr [r12]
mov   rbx, qword ptr [r13 + rdx*8]
```



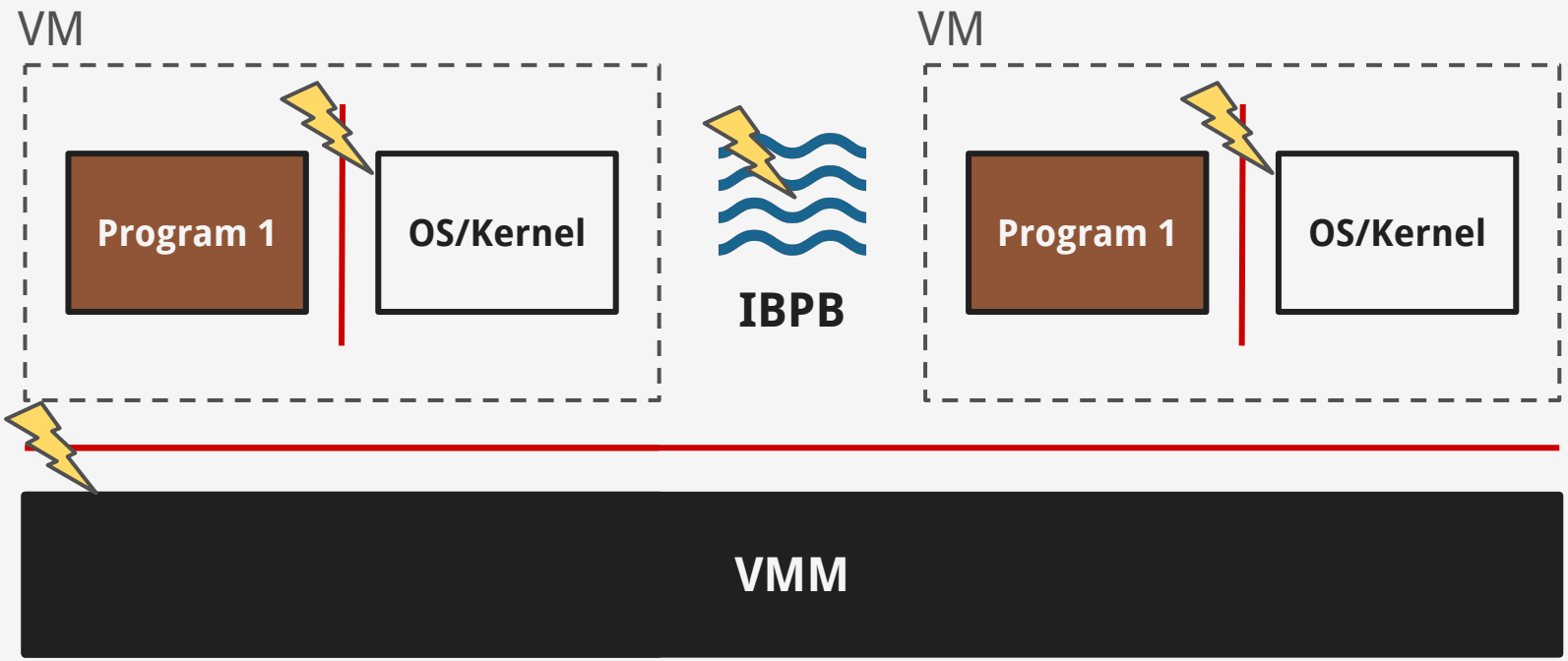
Analyze Kernel


call [rax]
disclosure gadget
secret
Shared Mem
Memory


Analyze Kernel
Break KASLR
Locate Shared Mem
Locate /etc/shadow
Leak /etc/shadow
root:6

DEMO TIME!

What security boundaries are affected?


VM
Program 1
OS/Kernel
IBPB
VM
Program 1
OS/Kernel
VMM

	IBPB Race	eIBRS Race
	✅	✅
	✅	

Emerald Rapids

Rocket Lake

Sapphire Rapids

Coffee Lake

Cascade Lake

Alder Lake

Raptor Lake Refresh

Lunar Lake

Sandy Bridge

Kaby Lake

Coffee Lake Refresh

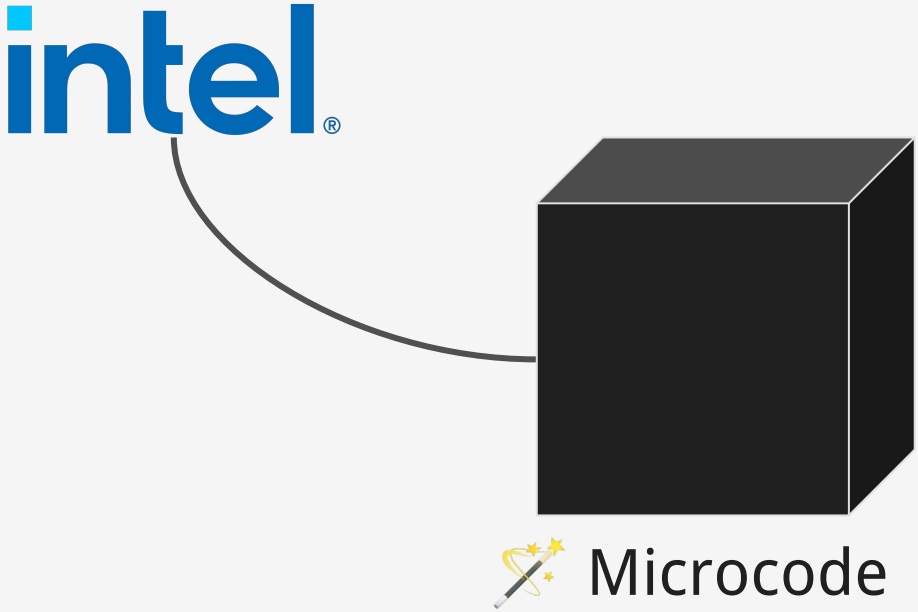
Comet Lake

Raptor Lake

Arrow Lake

2012 2017 2018 2019 2021 2022 2023 2024

How do we fix it?



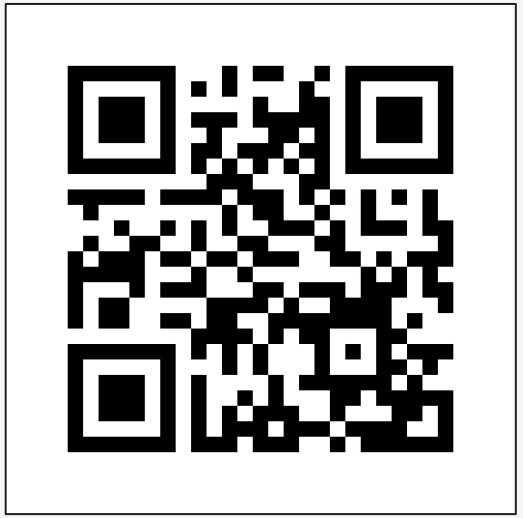
🪄 Microcode

What does it do?

🤷

Conclusion

- Race condition, undermining the long trusted eIBRS mitigation
- Spectre: a cat n’ mouse game between offense/defense.
- We must assess “black box” mitigations.
  - Type confusions
  - Race conditions
  - Use of uninitialized μarch buffers



comsec.ethz.ch/bprc